

Deadpan.

Make the moment
last considerably too long.

A keyboard-native editor for timing and attention.

Editable beats. Awkward pauses. Escalating repeats.
Local AI holds. One deliberate export.

RUST-FIRST / macOS / APPLE SILICON

FULL PRODUCT SCOPE · VERSION 1.0 · 20 SEPTEMBER 2026

Implementation-agent design package. Architecture, interaction contracts, runtime selection, packaging, and release acceptance criteria.

Contents

33 sections covering the complete product. The companion agent handoff identifies the first implementation gates; the keyboard reference summarizes the default editing language.

1. Product decision	3
2. Experience and success criteria	4
3. Vocabulary and primitive hierarchy	5
4. Exact time, duration, and synchronization	6
5. Document model and structural identity	8
6. The editing language	9
7. Keyboard design	12
8. Operation catalogue and editable recipes	15
9. Interface and information hierarchy	17
10. Audio model and signal flow	19
11. Transcription, analysis, and target tracking	20
12. Local AI holds: behavior, not a demo button	22
13. AI implementation selection and performance qualification	24
14. Runtime and model packaging	26
15. YouTube and local import	27
16. Media engine and native acceleration	29
17. Shared render plan and evaluation	30
18. Processes, jobs, and GPU scheduling	31
19. Caching and incremental computation	33
20. Projects, history, and recovery	35
21. Command API, CLI, and extensibility	36
22. Automatic YouTube-oriented rendering	38
23. Repository and dependency decisions	40
24. Workspace and component contracts	43
25. Performance requirements and instrumentation	44
26. Test strategy and fixture contracts	45
27. Security, licensing, and distribution	47
28. Failure behavior and non-negotiable invariants	48
29. Requirements traceability	49
30. Implementation workstreams and delivery gates	50
31. End-to-end worked edit	51
32. Resolved choices and remaining empirical questions	52
33. Primary sources and research notes	54

Reading note. This is a design, not an implemented application. Performance figures marked as targets are unmeasured requirements. Repository assessments are source-level; backend and model choices require qualification on the target Mac. Reference hardware: M5 Max, 128 GB unified memory. MLX is optional. Deadpan is a working name pending distribution-name clearance.

1. Product decision

Build a **structural editor for timing and attention**, not a conventional multitrack editor with keyboard shortcuts added afterward.

The main document is a sequence of **beats**. A beat can play source footage, hold a moment, repeat another beat, retime it, or contain a group. Camera moves, audio treatments, cutaways, captions, and sound events attach to these structures. Most jokes are combinations of a few ordinary operations, not special-purpose effects that flatten the edit.

The defining interaction is:

Navigate to a moment → select a meaningful object → apply an operation → adjust its parameters → audition → repeat elsewhere.

Examples: repeat a word three times; insert 1.5 seconds of silence after a sentence; creep toward a selected face during the pause; replay a reaction with progressively louder breaths. The editor should make these actions easier than moving clips around.

1.1 Decisive technical choices

Concern	Decision
Application	New Rust workspace; native desktop application, not a browser UI.
UI	egui / eframe with wgpu on Metal, custom preview and timeline widgets.
Editing model	Typed, non-destructive beat tree, attached events, stable anchors, immutable revision snapshots.
Interaction	Vim-inspired motions, text objects, operators, counts, registers, macros, and dot-repeat.
Media	Pinned FFmpeg libraries through a narrow Rust adapter; VideoToolbox acceleration where qualified.
Rendering	One render-plan implementation, GPU compositor, and audio DSP graph shared by preview and final rendering.
Audio	48 kHz internal mix, floating-point DSP, native audio output; sample-accurate audio-only edits.
AI	Local worker protocol with replaceable backends; benchmark small LTX-Video and current LTX MLX implementations before selecting shipped defaults.
Project storage	SQLite is authoritative inside a .deadpan directory package. Original and accepted generated media remain separate from evictable caches.
Import	Bundled yt-dlp, its required JavaScript support, and FFmpeg. Local import is equally first-class.
Output	One Render action; automatic YouTube-oriented encoding from project and source characteristics.

Concern	Decision
Distribution	Signed, notarized, self-contained application; app-managed model downloads, with an offline pack distribution path.

The UI stack is a design choice supported by egui's native winit/wgpu integration, not a claim that it removes the need to engineer a native-quality interface. [S01]

1.2 What “complete” means

The finished product includes the editing language, all operations in this document, source organization, transcription and analysis, tracking, AI holds, packaging, offline use after assets are installed, recovery, migration, performance instrumentation, accessible keyboard navigation, export verification, documentation, and automated tests.

Implementation workstreams are ordered by dependencies. They are **not** permission to ship the first few workstreams as the completed product. AI hold generation, reliable export, and clean-machine installation are release gates, not decorative integrations.

Deliberately outside the product identity: cloud accounts, collaboration servers, automatic YouTube publishing, a full digital audio workstation, a general node-compositing application, 3D scenes, face replacement, voice cloning, and an autonomous “make it funny” agent. These are not necessary to satisfy this complete specialized product.

2. Experience and success criteria

2.1 Primary session

1. Create a project or open an existing `.deadpan` package.
2. Paste a YouTube URL or import a local file. Start browsing as soon as local media is available; background analysis must not block editing.
3. Search the transcript, step through words and shots, mark interesting reactions, and insert selected footage into the sequence.
4. Apply pauses, repeats, reframing, sound manipulation, and cutaways through composable commands.
5. Generate live holds locally while continuing to edit. Audition and explicitly accept a generated candidate.
6. Render the current committed revision to an automatically configured MP4.

A successful session needs no Terminal, Homebrew, Python installation, model-server configuration, manual FFmpeg installation, or knowledge of encoding parameters.

2.2 Acceptance examples

Word escalation. The user can navigate to a transcript word, type `3riw`, then add a +3 dB and +0.08 scale increment per iteration. The result is one editable repeat structure with three total plays. Changing the count to four preserves the selected source and the progression.

Awkward interruption. At a frame boundary in speech, `3,h` inserts a 1.5-second freeze with silent audio. Playback resumes at the original next frame with the original subsequent speech, not a duplicated or regenerated sentence.

Live stare. An AI hold replaces only the inserted hold's visual provider after acceptance. Rejecting it, regenerating it, or using a freeze instead cannot move downstream content or alter the audio.

Reaction. A marked reaction can replace the picture for a selected interval while the original audio continues. It remains attached to the host beat when that beat moves.

Portable project. A project containing accepted generated clips opens and renders offline on another supported Mac without downloading the generation model again.

2.3 Responsiveness policy

Ordinary edits must feel immediate regardless of whether analysis, downloading, or inference is running. A generated hold may take time, but inserting the hold must not. The user is never required to wait for transcription, a face detector, a proxy, or a model to complete before cutting existing footage.

The program reports actual job state and measured durations. It must not label a backend “real-time” based on someone else's GPU benchmark or display invented progress percentages.

3. Vocabulary and primitive hierarchy

Use the same terms in the UI, command language, code, schema, and help.

Level	Primitive	Meaning
Measurement	Frame, sample, source timestamp	Distinct typed coordinates; never interchangeable floating-point seconds.
Reference	Asset	Immutable original or generated media plus stream metadata and content identity.
Reference	Span	A half-open interval in a specified coordinate system.
Reference	Anchor	A stable attachment to a source moment, beat location, occurrence, or explicit sequence time.
Editing	Beat	A contiguous structural unit with a duration and an audiovisual recipe.
Editing	Sequence	Ordered child beats whose durations concatenate.
Editing	Repeat	A child beat or group, a total-play count, gaps, and iteration-dependent modifiers.
Editing	Hold	Inserted time with independent visual and audio policies.
Editing	Retime	A duration and mapping from output time into child time.
Attention	Target	A normalized point, region, or tracked region used for framing.
Attention	Envelope	A typed parameter varying over time or repetition index.
Composition	Attachment	A sound, cutaway, caption, or treatment attached to a host interval.
Authoring	Gag	A named group or parameterized recipe built from the same primitives.

Level	Primitive	Meaning
Control	Command	Typed editing intent resolved against a selection and document revision.
Storage	Revision	An immutable committed state and its transaction identity.
Computation	Artifact	Content-addressed output of analysis, generation, proxying, or rendering.

Beat does not mean musical beat. It means an editable moment. Musical synchronization is not implied.

Keep the primitive set small. A “nervous replay” is a repeat plus a gap and camera/audio envelopes; it is not another core node type. Do not create a new database table or rendering subsystem for every named gag.

4. Exact time, duration, and synchronization

4.1 Coordinate systems

The project uses a fixed rational presentation frame rate, such as 30000/1001, chosen automatically when its first primary video is inserted. Source streams retain their actual timestamp time bases and presentation timestamps, including variable frame timing. Audio processing uses 48,000 samples per second.

Define separate newtypes, conceptually:

```
struct ProjectFrame(i64);
struct AudioSample(i64);
struct SourceTimestamp { ticks: i64, time_base: Rational }
struct FrameRate { numerator: u32, denominator: u32 }
struct FrameRange { start: ProjectFrame, end: ProjectFrame }
```

All authored timeline ranges are half-open: $[start, end)$. A five-frame beat contains exactly five output frames. An insertion at boundary f occurs before the frame that previously occupied f .

Do not accumulate durations using $f32$, milliseconds, or repeated rounding. Rational conversions use checked wide intermediates. Reject overflow and invalid rates. Represent fractional rates exactly; never substitute 30 for 29.97.

4.2 Frame-to-sample mapping

The audio boundary corresponding to project frame f is:

$$B(f) = \text{round_even}(f \times 48000 \times \text{fps.denominator} / \text{fps.numerator})$$

The audio interval for frames $[a, b)$ is $[B(a), B(b))$. Compute both boundaries from the common origin; do not separately round each beat's duration and add it. This prevents drift when thousands of fractional-rate edits are concatenated.

Audio-only attachments may start at arbitrary samples. Linked audiovisual structural edits snap to project frame boundaries; audio is cut at the mapped sample boundary. Whisper word boundaries are suggestions, not a replacement for this clock.

4.3 Source and proxy mapping

A source-frame index records presentation order, original PTS, duration, keyframe references, and decode hints. Decoding order must not be confused with presentation order. Proxies retain a mapping to original source frame identity; they never become the authoritative editorial time base.

Source sampling uses presentation timestamps. At each requested project-frame center, choose the source frame whose presentation interval contains that time; hold the adjacent valid endpoint only when explicitly allowed. Rate conversion defaults to deterministic frame selection, not optical-flow invention. Provide creative optical-flow interpolation only as an explicit materialized treatment, never an implicit import correction.

Honor rotation, sample aspect ratio, clean aperture where available, negative timestamps, audio offsets, and codec delay. Non-zero source timestamps normalize through an explicit origin record, not by forgetting synchronization metadata.

4.4 Duration rules

Node	Output duration
Source	Explicit project-frame duration and mapping to a valid source span.
Sequence	Sum of child durations.
Hold	Exactly the requested integer number of project frames.
Repeat	$\text{plays} \times \text{child_duration} + (\text{plays} - 1) \times \text{gap_duration}$.
Retime	Explicit output frame count; mapping endpoints must be valid within child time.
Attachment	Does not change host duration; tails follow explicit ownership policy.

$\text{plays}=3$ means **three plays total**, not the original plus three copies. There is no trailing repeat gap. A repeat gap is represented by a Hold recipe, so its picture and audio behavior remain explicit.

Text time inputs round once to project frames with ties-to-even and display the resolved duration. Inputs include 12f, 250ms, 1.5s, and 01:02.500. Unitless positional numbers are only allowed where the command has a documented dimensionless parameter, such as repeat count.

4.5 Structural edits and micro-edits

Linked word selections snap outward to the smallest enclosing project-frame interval, and the UI shows both the transcript boundary and resolved cut. This can include a few extra milliseconds; the waveform and audio-only selection let the user correct it. Never pretend a video frame can be cut in half.

For a syllable shorter than a frame, use an audio-only repeat/attachment over a chosen picture. The command must display that the operation is audio-only. A media-role selector is explicit in command state: linked, video, or audio; linked is the default.

5. Document model and structural identity

5.1 Authored nodes

Store a typed tree of beat nodes with stable IDs and a root Sequence. Assets may be referenced many times; authored child nodes have one structural parent. Repeats instantiate a child without copying its authored definition for each play.

```
ProjectDocument
  schema_version, project_id, revision_id
  presentation_basis: dimensions, frame_rate, color_policy
  root: NodeId
  nodes: NodeId -> BeatNode
  assets: AssetId -> AssetRecord
  targets: TargetId -> Target
  attachments: AttachmentId -> Attachment
  annotations, named_ranges, saved_gags, registers
```

BeatNode variants are Source, Sequence, Hold, Repeat, and Retime. Each carries a label, typed effects, and editorial metadata. A generated hold is a Hold whose visual provider refers to an accepted artifact; it is not a different kind of timeline clip.

The source node describes video and audio selections independently, with a link relation and synchronization offset. Splitting linked media preserves this relation. Audio-only and still-image sources are valid; audio-only insertion uses an explicit blank/held picture policy.

5.2 Occurrences and repeat overrides

An authored NodeId is insufficient to identify the second appearance of a nested repeat. Use an InstancePath: structural IDs plus repeat iteration identities. Default edits to a Repeat's child affect every iteration. The inspector clearly labels **All plays** versus **This play**.

An edit to one occurrence creates an explicit sparse override for that iteration. Changing count retains overrides for surviving iterations, archives removed overrides in undo history, and never silently applies an old override to a different iteration. Reordering or inserting iterations uses stable iteration IDs, not a positional array index as identity.

explode converts a repeat into an ordinary sequence, preserving all current pictures, sound, timing, and overrides. It remains undoable. Copy/paste creates new authored IDs while sharing immutable source media; it does not introduce hidden live links between independent copies.

5.3 Anchors

Every attachment declares an anchor coordinate space:

- **Source anchor:** a source PTS or source sample; useful for transcript tokens and selected reactions.
- **Local anchor:** a position inside an authored beat; follows that beat when moved.
- **Occurrence anchor:** a local point in one concrete repeated occurrence.
- **Sequence-time anchor:** fixed project time, only for deliberately pinned events.

A boundary anchor also has left/right insertion bias. Inserting exactly on a left-biased anchor leaves it before the insertion; right-biased follows the original right-hand content. Persistent attachments default to host-local anchors, not absolute timeline seconds.

Deleting an anchor's host either deletes its owned attachment or marks it unresolved according to an explicit attachment policy. No silent retargeting to whatever now happens to occupy that timestamp.

5.4 Effects and ownership

Effects declare both their evaluation space and order. An effect inside Repeat is evaluated anew per play; an effect on Repeat spans the entire repeated passage. A target track is evaluated in source time, then mapped through local retiming and editorial framing.

Moving a group moves its owned attachments. Ripple deletion removes owned events inside the removed content and trims intersecting host-local events. Sequence-pinned events remain fixed and are visibly marked. Duplicating a group duplicates its attachments unless explicitly marked shared, such as one persistent music bed outside the group.

Validate: no cycles; one structural parent per authored child; positive durations except explicitly empty sequences; valid assets; bounded ranges; finite effect parameters; valid occurrence references; no NaN values. Protect against maliciously deep documents with a generous documented depth limit and iterative traversal.

6. The editing language

6.1 One path for every input

Keyboard, menus, inspector controls, CLI, macros, and a future agent client all produce the same typed Command. No UI widget mutates the document directly.

```
Input gesture
-> Parsed command + selector
-> Resolve against revision and focus
-> Validate / calculate effect preview
-> Atomic transaction
-> New revision + invalidations + job requests
```

Commands are semantic. A macro records “repeat the inner word three times,” not a stream of screen coordinates or key events. The command parser cannot execute arbitrary shell commands.

6.2 Selectors

A selector resolves to a point, range, set of ranges, node, or occurrence. It includes media role, coordinate space, and revision. Supported selectors include current visual selection, inner/around text object, relative motion, named mark range, asset/source range, explicit node ID, and transcript search result.

At an ambiguous boundary, choose the right-hand object except at document end, where choose the final object. Display the selected range before executing an operator that is still pending. A nonexistent word/pause/shot produces a useful no-op message; it must not silently fall back to the entire project.

Discontiguous ranges are supported through search results and explicit selection commands. Apply edits from later time to earlier time within one atomic transaction. Dot-repeat preserves the semantic selector and parameters, not the previously resolved absolute timestamps.

6.3 Operator semantics

Operation	Contract
Delete	Ripple-delete linked content. Role-only deletion removes that role over the same interval without shifting the other role or the timeline.
Yank	Store an editable substructure and its owned attachments in a register.
Paste	Insert a copy at an explicit boundary; preserve source references and internal relationships.
Repeat	Wrap selection, retain original total duration as child, set total plays, and keep count editable.
Split	Split at cursor without changing rendered output or total duration.
Hold	Insert new time at a boundary; never stretch original speech unless explicitly requested.
Replace picture	Add a cutaway over host duration; keep host sound unless requested otherwise.
Retime	Map a selected child into explicit new duration; choose pitch policy.
Group / ungroup	Change structure without changing rendered output.
Lift	Replace selected linked content with a same-duration blank/silent hold; no ripple.
Slip	Shift source in/out equally, retaining output position and duration.
Roll	Transfer duration across an adjacent boundary, retaining combined duration.
Trim	Change one source edge with explicit ripple or overwrite policy.

Operators on a partial nested structure split only the required boundary nodes and wrap the resulting fragment. They do not flatten unrelated repeat groups or convert everything to rendered media.

6.4 Command-line grammar

Use `:<verb> [positional arguments] [name=value ...]`. Strings use quoted JSON-style escapes. Booleans are `true/false`. Typed units include seconds, milliseconds, frames, dB, semitones, and normalized scale. Completion is schema-driven and describes the default selector before execution.

```

:hold 1.5s video=freeze audio=silence
:hold 1.5s video=ai audio=silence
:repeat 3 gap=120ms gain-step=3dB zoom-step=0.08
:zoom 1.35 target=face:2 curve=step
:creep from=1 to=1.4 target=current
:gain +6dB
:retime 0.75 pitch=preserve
:cutaway register=r audio=keep
:tail 400ms effect=reverb
:trim edge=out delta=-3f mode=ripple
:slip +5f
:roll +2f
:select role=audio
:group name="the uncomfortable answer"
:render

```

`retime 0.75` means playback at $0.75\times$ source speed; the UI also reports the resulting frame duration. `zoom-step=0.08` means an additive change in scale per play, not an 8% multiplicative progression. Add `progression=multiply` explicitly when desired. Durations and resolutions always show the final quantized result.

Every editing command exposes `--dry-run` in CLI/JSON form, returning resolved selectors, prospective duration changes, affected IDs, warnings, and required jobs. Interactive previews may be immediate without a confirmation dialog; dry-run exists for observability and agent safety, not as a mandatory extra click.

6.5 Media-role operations and boundary cases

Linked edits are the default. A linked delete removes time; an audio-only delete replaces that role with silence over the existing interval; a video-only delete removes that picture contribution, exposing an underlying attachment or the project background. Neither role-only deletion moves later content. A separate explicit `audio-shift` command is required to ripple audio independently.

An audio-only repeat creates an attached, sample-timed event with the calculated repeated duration and mutes the underlying host audio over that event. It does not insert picture time. If the event exceeds available host duration, require an explicit `extend=hold` or `overflow=trim` policy; do not silently discard repeats. A video-only repeat similarly maps repeated picture over a fixed host interval, with an explicit `fit/trim/hold` policy. Linked Repeat remains the structural duration-changing operation.

At project start, freeze insertion uses the first available picture; at project end, it uses the final one. In an audio-only project it uses the project background. A zero-duration time insertion is a no-op with a message; a negative duration is invalid. Inserted time inherits the surrounding group's ownership but does not capture unrelated sequence-pinned events. When an insertion lies inside a source beat, split that beat at the boundary and preserve its exact source mapping on both sides.

7. Keyboard design

7.1 Modes and focus

Modes: **Normal**, **Visual**, **Operator pending**, **Command/text entry**, **Camera**, and **Trim**. Source and Sequence are editor contexts, not additional unrelated keymaps. Context appears beside mode in the status line.

Normal-mode bindings apply only when no text field or IME composition has focus. Standard macOS text editing, copy/paste, accessibility navigation, and Command-key menu shortcuts remain intact. Escape leaves a transient mode or cancels an uncommitted parameter preview; it does not discard already committed edits.

Use a declarative binding trie. Prefixes have no execution timeout; a help popup may appear after a delay, but typing slowly cannot change a command's meaning. Reject ambiguous bindings at configuration load with the conflicting paths shown.

7.2 Navigation and transport

Keys	Meaning in Normal / timeline Visual context
h, l	Previous / next project frame. Count supported.
j, k	Next / previous beat at current structural depth.
w, b, e	Next word start, previous word start, current/next word end.
W, B	Next / previous sentence start.
]c, [c	Next / previous edit boundary.
]s, [s	Next / previous detected source shot boundary.
]p, [p	Next / previous silence interval.
gg, G	Start / end of current sequence or source.
m + letter	Set mark.
' + letter	Jump to mark; retain precise timeline position.
Ctrl-o, Ctrl-i	Back / forward in jump history.
/, n, N	Search transcript/tags, next / previous result.
Space	Play / pause.
Shift-Space	Audition selected range in a loop with short context handles.
Enter, Backspace	Drill into selected group / return to parent.
Tab, Shift-Tab	Cycle focus among visible panes.

For endpoint motions, the cursor is a boundary between frames; the viewer shows the frame to its right, or the preceding final frame at project end. Thus “insert at the cursor” always has one meaning. A small boundary indicator avoids making a displayed still appear to be the left-hand frame accidentally.

7.3 Text objects

After an operator or in Visual mode, **i** means tight/inner and **a** means with nearby context.

Object	Meaning
iw, aw	Word; word with adjacent non-speech handles.
is, as	Sentence; sentence with surrounding pause handles.
ip, ap	Silence interval; silence with short adjacent audiovisual handles.
ib, ab	Current beat; beat plus its owned temporal attachments.
ig, ag	Current group contents; group as an owned structural unit.
iS, aS	Detected shot; shot with transition handles when available.

For word/sentence “around” handles, allocate at most half of each adjoining non-speech interval, capped at 80 ms on each side. Never knowingly include neighboring speech simply to meet the cap. If analysis is unavailable, explain that this object is not ready; frame/beat operations remain available.

ib and **ab** have identical primary-picture time ranges but differ in attachment selection. Whole-project selection is the explicit `:select all`, not an easy-to-mistype single-letter object.

7.4 Editing operators and registers

Keys	Meaning
v	Start / finish a time-range Visual selection.
d + object/motion	Ripple delete; dd deletes current beat.
y + object/motion	Yank; yy yanks current beat with owned attachments.
r + object/motion	Repeat; default two total plays. rr repeats current beat.
3riw	Make the inner word play three times total.
p, P	Paste after / before the current beat; at a Visual selection, replace it.
s	Split linked content at the cursor.
x	Ripple-delete one project frame; count supported.
u, Ctrl-r	Undo / redo.
.	Repeat last committed semantic edit with the new current selector.
" + register	Select named register for next yank/delete/paste.
q + register, q	Start recording semantic macro / stop recording.

Keys	Meaning
@ + register	Execute macro; count repeats the macro.
:	Command line.
?	Searchable contextual key help.

A count before *r* is total plays. Counts before motions indicate motion distance. Reject two conflicting counts such as *3r2iw*; do not multiply them silently. A count of one produces a one-play repeat, which may be normalized away only if there are no repeat-specific parameters to preserve.

While recording a macro, bare *q* stops recording and cannot start another macro. Registers hold typed content or a macro and show that type; incompatible use produces an error. Macro execution is one undoable transaction, validates all operations first, and has recursion and instruction limits. Do not allow macro execution to invoke arbitrary processes.

7.5 Comma-leader editing vocabulary

These bindings are mnemonic accelerators for ordinary commands. All actions also exist in the palette.

Keys	Default action
,h	Insert a 0.5 s freeze hold with silence. Count scales duration: 3,h = 1.5 s.
,a	Insert the same hold and request an AI candidate. Same count semantics.
,z	Punch in to 1.35× on the selected target.
,c	Creep from current framing to 1.35× over selection.
,m	Mute selected audio without removing time.
,r	Open reaction/register picker for a picture-only cutaway.
,e	Apply escalating-repeat recipe and focus its parameters.
,b	Replace selected audio with a generated bleep.
,t	Add a reverb tail using available handles; focus duration.
,g	Group the selection as a named gag.
,f	Enter Camera mode.
,v	Enter Trim mode.

+ / - in Normal mode change selected audio gain by ± 3 dB. With no explicit selection, they affect the current beat, never the entire master bus. Camera mode owns its own + / - meanings. The status line shows the exact scope and resulting value before and after each change.

7.6 Camera mode

The viewer shows numbered targets: detected regions, saved targets, center, and corners. Digits select a target. h/j/k/l move framing center by 1% of the uncropped source dimension, uppercase variants by 5%; +/- multiply scale by 1.05 or its reciprocal; counts multiply repetitions of that operation. f refreshes target selection; r resets the temporary framing; Enter commits once; Escape restores the entry state.

Region creation is fully keyboard-operable: choose center, then width and height fields with Tab, adjust using arrows/counts, and commit. The mouse may drag the same rectangle but is never required. Switching subjects during a creep creates an explicit target/envelope change, not a new hidden crop coordinate system.

7.7 Trim mode

Tab cycles in, out, slip, and roll; h/l move the selected boundary by frames; Shift gives ten-frame steps. r toggles ripple versus overwrite where meaningful. The viewer displays outgoing/incoming boundary frames and the audio waveform. Enter commits one transaction; Escape restores the original. Invalid source extension clamps to available handles with a visible explanation.

8. Operation catalogue and editable recipes

The catalogue below defines the complete creative surface. Every operation must support undo, keyboard control, serialization, preview, export, and parameter adjustment after insertion.

8.1 Time and delivery

Operation	Construction / behavior
Dead air	Hold + true digital silence. Adjustable duration; preserves following speech.
Frozen stare	Hold of exact source frame; zoom and audio remain independently editable.
Living stare	Same Hold using accepted AI video with generated audio discarded.
Micro-loop	Repeat a selected low-motion fragment inside a Hold, with explicit seam treatment.
Ping-pong hold	Forward/reverse fragment with endpoint duplication removed; no interpolation required.
Word / syllable stutter	Repeat selected linked or audio-only span; optional inter-play hold.
Escalation	Repeat with per-play gain, zoom, speed, or gap progression.
False start	Play a short prefix, pause, then restart a longer source span.
Interrupted answer	Cut to a hold/reaction before a phrase resolves, then resume or omit its ending.
Callback	Paste an earlier gag/reaction from a register or named range.
Reverse hiccup	Brief reversed span followed by ordinary forward content.
Slow delivery	Retime with pitch-preserving or tape-speed sound, explicitly selected.

8.2 Attention and picture

Operation	Construction / behavior
Smash zoom	Step change in scale/center on an anchored target.
Slow creep	Framing envelope over a beat, hold, or entire repeated group.
Escalating crop	Per-play framing increment without duplicating authored clips.
Reaction cutaway	Picture-only replacement from another source/register; original audio continues.
Reaction ping-pong	Alternating picture attachments, preserving or switching sound explicitly.
Off-center stare	Deliberately awkward framing retained as a reusable target preset.
Freeze a detail	Hold/reframe a hand, eyes, object, or other region; no face-only assumption.
Black-frame punctuation	Hold with black picture and chosen silence/sound policy.
Delayed caption	Text attachment with local delay or per-play reveal; editable and optional.
Abrupt return	Remove/step out of framing at a chosen boundary; hard cut is the default.

8.3 Audio

Operation	Construction / behavior
Selective emphasis	Gain envelope on a word, breath, click, or arbitrary waveform range.
Sudden silence	Mute envelope or silent inserted Hold; distinct from room tone.
Room tone	User-approved non-speech source loop with small crossfades; never silently substitute for silence.
Hanging tail	Delay/reverb output continues into a pause under an explicit tail policy.
Bleep	Synthesized tone replacing or overlaying a selected interval.
Audio lag	Offset an audio role relative to its picture while retaining an explicit link offset.
Premature sound	J-cut: following audio starts before the picture changes.
Lingering sound	L-cut: outgoing audio continues over another picture.
Saturation	Intentional nonlinear drive before the master limiter; not accidental integer clipping.
Pitch shift	Semitone-based change independent of duration when pitch-preserving processing is used.
Bed drop	Attached ambience/music bed abruptly cuts at a selected anchor.
Wrongly triumphant sting	User-owned sound or bundled original synthesized sting attached to an otherwise quiet moment.

A user supplies copyrighted sound clips they are entitled to use. Bundle only original or appropriately licensed effects; do not distribute an unlicensed “meme sounds” archive.

8.4 Starter recipes

Ship parameterized recipes including **The Long Answer** (sentence + silent live/freeze hold + creep), **One More Time** (three plays with a shorter gap each time), **Are We Done?** (reaction cutaway while the previous audio tails off), **The Escalator** (repeat with increasing gain/scale), **The Non-Sequitur** (hard cut to registered detail and immediate return), and **Nothing Happens** (room tone cut to true silence while the picture keeps holding).

Each recipe expands to ordinary nodes and attachments with named exposed parameters. The recipe definition is versioned. An inserted gag pins that version and stores its parameters; upgrading a recipe never mutates existing projects. Users can save a modified group as a new local recipe, inspect its expansion, or detach it from its template.

Seeded variation is allowed for intentionally irregular pauses or slight framing changes. Store the seed and resolved envelope values. Do not generate new randomness on every playback or export.

9. Interface and information hierarchy

9.1 Single-window layout

The default workspace has a large preview, a compact structural timeline beneath it, a transcript/source browser on the left, and an inspector on the right that appears only when useful. A thin bottom command/status bar is always visible.

Project / source name	Jobs	Render
+-----+-----+-----+		
Sources /		
transcript /	Video preview	
named moments		
+-----+-----+-----+		
Current sequence: beat blocks + waveform	Parameters	
Attached picture / sound / caption events	or targets	
+-----+-----+-----+		
NORMAL · Sequence · linked · 00:18.400 · selected word		
pending command / feedback / contextual keys		
+-----+-----+-----+		

The picture should dominate. Avoid permanent professional-editor chrome, unused tracks, tiny toolbars, and a node graph. Advanced controls live in a focused inspector, palette, or drill-down; they remain keyboard-accessible.

9.2 Timeline at several scales

The same underlying sequence supports three presentations: **story view** for beats and gags, **timing view** for frames/waveforms, and **detail view** for envelopes and attachments. These are zoom/detail levels, not separate documents.

Repeat groups display one bracketed object with iteration ticks and count. Expanding it reveals occurrences without exploding its structure. Holds display their duration and visual/audio policies. All candidates have a separate state badge; the committed picture provider remains visible.

Waveforms use a multiresolution min/max pyramid. Thumbnail strips are virtualized. A long timeline must not create one UI widget, texture, or decoder per frame or repeat occurrence.

9.3 Source browser and transcript

Sources support fuzzy search, tags, thumbnails, media status, and named moments. The transcript follows source timing; the Sequence transcript shows words through the edit, including repeated occurrences. Both panes support keyboard search and selection.

Transcript correction edits text/annotation, not audio. An explicit replace/transcript-edit mode may delete the corresponding timed ranges, but normal typing in a transcript never unexpectedly cuts video. Low-confidence words are visibly approximate. The user can drag or keyboard-adjust their boundaries without waiting for retranscription.

Allow filtering named moments by tags such as `reaction`, `breath`, `detail`, or user-defined strings. Tags describe editorial intent, not automatically inferred emotions or identity.

9.4 Inspector and discoverability

The inspector exposes only parameters relevant to the selected node or attachment: repeat count and gaps; hold duration/provider; framing target; gain; retime policy; tail duration. Every field shows units and supports direct entry, arrows, fine/coarse adjustment, reset, and reference to its command name.

A prefix popup teaches commands while leaving the frame visible. A searchable palette lists all actions, their shortcuts, current scope, and whether the required analysis is available. The UI must not hide disabled actions without explaining why.

9.5 Audition and variants

Audition loops include a default 500 ms lead-in and 750 ms follow-through, bounded by the sequence. The user can change context through a command. Switching candidates or toggling before/after preserves the same audition time window.

Temporary parameter changes render as an explicit preview revision. Enter commits one transaction; Escape restores. A/B comparison compares the last committed revision to the proposed one. Playback restarts from the audition start only when requested; routine edits should not constantly jump the cursor.

9.6 Accessibility and native behavior

Support keyboard-only onboarding, file dialogs, target picking, model installation, error recovery, and export. Respect UI scaling, reduced motion, high contrast, and standard focus indicators. Expose meaningful accessibility labels and values through the chosen UI integration; a painted timeline is not automatically accessible merely because the framework supports accessibility. [S01]

Provide a textual accessibility representation of the selected beat, duration, source, attachments, and available operations. Do not require color discrimination to distinguish pending, accepted, failed, or muted states. Respect native text composition and non-US keyboard layouts; bindings are logical keys by default, with optional physical-key bindings.

10. Audio model and signal flow

10.1 Independent but linked sound

Picture and sound are linked by default but never conflated in the data model. Every temporal operation declares whether it changes video, audio, or both. A silent hold creates new silent time; muting a source interval keeps its existing time; removing audio leaves picture duration unchanged.

The internal mix is floating-point stereo at 48 kHz. Preserve original multichannel media and downmix using an explicit layout-aware matrix for the YouTube stereo mix. Mono is centered. Do not guess channel ordering from channel count alone.

10.2 Fixed default effect order

For each audio voice:

```
source decode / source offset
-> resampling and time/pitch mapping
-> edge fades
-> clip gain envelope
-> EQ / filtering
-> saturation
-> pan / spatial balance
-> delay / reverb sends
-> group bus
-> master safety limiter
-> output conversion
```

Expose reordering only within a small documented set of treatment stages, rather than an unrestricted audio plugin host. Store effect order explicitly. A recipe that depends on distortion before a gain step must serialize that order rather than rely on UI placement.

10.3 Loudness policy

Do not normalize each repeated word independently. That would destroy the intended escalation. Do not automatically level every pause, breath, or quiet reaction up to a target.

Default: preserve source level, apply authored gain, and enforce a final **−1 dBTP ceiling** using a tested oversampled true-peak limiter. This ceiling is a Deadpan design default, not a claim that YouTube mandates that value. Report master integrated loudness and limiting reduction informationally; there is no mandatory −14 LUFS normalization pass.

Optional source-level matching applies one constant gain per source, derived from analyzed speech, and is an explicit project edit. It must not introduce automatic per-word gain riding. Monitoring volume is independent of exported gain.

10.4 Clicks, tails, silence

Use short edge fades, normally 2 ms and shortened for tiny fragments, to prevent unintended discontinuities. Fades occur inside the allocated interval and do not lengthen a beat. Hard discontinuity remains an explicit creative option.

An inserted silent hold defaults to suppressing direct audio and incoming effect tails over its interval. A hanging-tail recipe instead permits selected sends to spill into it. These policies appear as *silence*, *room-tone*, *tail*, or *custom*, not an ambiguous mute toggle.

Room tone is chosen from non-speech material with a visible source range and audition. Speech detection can suggest candidates but cannot guarantee absence of quiet words. The user can choose a different range. Loop crossfades stay within the allocated Hold duration.

10.5 Time and pitch

Use a qualified native time-stretch implementation behind a narrow adapter; **SignalSmith Stretch** is the selected initial library candidate, with its exact distribution license recorded at pinning. Its author documents independent time/pitch processing. [S02]

Support tape-speed, pitch-preserving stretch, fixed pitch shift, reverse, and variable rate. Account for algorithmic latency and preroll explicitly. The same DSP implementation processes preview and offline export. If an extreme transformation cannot run interactively, materialize that audio node into a cache and display its state; do not use a different-sounding “close enough” algorithm for export.

11. Transcription, analysis, and target tracking

11.1 Required analysis services

Implement local waveform/peak generation, shot-boundary proposals, silence/VAD intervals, transcription with word timing, face/region proposals, and selected-target tracking. These services create versioned annotations and artifacts. They never directly change an edit.

Use `whisper.cpp` as the initial transcription runtime, Silero VAD or a measured lightweight equivalent for speech activity, and Apple Vision for target detection/tracking where appropriate. A Rust ONNX adapter is acceptable for small auxiliary models. [S03][S04][S05][S06]

Bundle a modest transcription model in the full installer or install it through the same model manager. Offer a higher-quality local transcription pack through an in-app choice, not a settings page of raw runtime flags.

11.2 Timing reliability

`whisper.cpp` describes its word-level timestamp support as experimental. Therefore word selection requires confidence display, waveform adjustment, and a tighter local alignment pass where available; it is not guaranteed sample-accurate just because a transcript contains timestamps. [S03]

For a selected word or sentence, an on-demand refinement job may improve boundaries in a short context window. It returns a proposal. Existing cuts and manually corrected timing remain pinned. Silence intervals are derived from VAD plus measured energy and can be edited manually.

A sentence object uses transcript punctuation and pause boundaries under a versioned segmentation rule. Shot detection proposes source-frame boundaries from visual change metrics. Neither analysis changes the project frame rate or inserts cuts by itself.

11.3 Tracking contract

A Target contains a source asset, source-time range, normalized region/point, per-frame or sparse tracked transforms, confidence, and manual overrides. Editorial targets do not require identifying the person by name.

Tracking starts from a user-selected region, stops at shot boundaries by default, and marks occlusion or loss rather than jumping to a different person. Interpolate short low-confidence gaps only under a bounded policy; longer gaps use a held manual position until corrected. Keyframe corrections invalidate only the affected tracking range.

Use tracking to stabilize framing, not to claim knowledge of emotion. “Uncomfortable” is a user-given gag label, not a face classifier output.

11.4 Scheduling and privacy

Analysis runs incrementally by source ranges, prioritizing the visible cursor neighborhood. Cache results by original media hash, model, settings, and implementation version. Store local speech/face analysis only in project/private caches. No cloud transcription, telemetry upload, or model-provider contact is necessary during analysis.

12. Local AI holds: behavior, not a demo button

12.1 Definition

An AI hold adds a short interval in which the existing scene appears to continue with minimal motion while its dialogue does not. The default intent is a plausible, quiet continuation of posture and environment, not inventing new dialogue, changing identity, changing location, or adding action.

The operation is two independent edits: **insert time** and **choose the picture provider for that time**. Only the second requires a model. The first commits immediately and renders correctly as a freeze even without a model installed.

12.2 Exact insertion contract

At timeline boundary f , let L be the last original picture to the left and R the next original picture to the right. Insert N project frames. Original frames before f remain unchanged; original content starting at f moves to $f+N$. The original audio is split at $B(f)$ and resumed at $B(f+N)$ without regenerating words.

The provisional picture provider is a freeze of L ; at project start, use R . The requested audio policy is recorded separately, defaulting to silence. The playhead and undo history reflect the inserted duration immediately.

For a **bridge hold**, condition on L , R , and short same-shot context where the backend supports it. Generate a path between these boundary states, then retain exactly N interior output frames. Do not insert copies of the conditioning endpoints in addition to the requested frames.

For a **one-sided extension**, condition on L and lead-in context. Use it primarily at the end of a shot/sequence, or mark the outgoing seam as unconditioned. A model supporting image-to-video does not automatically support a reliable return to R .

12.3 Provider capability manifest

Every provider reports machine-readable capabilities: image-to-video, short-video conditioning, first/last-frame conditioning, extension/retake, allowed frame-count formula, spatial multiples, supported aspect ratios, native generation rates, precision modes, memory estimate, color expectations, model license, and runtime version.

The planner rejects unsupported conditioning combinations before loading the model. Do not silently ignore an end-frame condition and present the result as a successful bridge.

LTX-Video documents multiple conditioning and extension workflows; capabilities still depend on the actual selected checkpoint and pipeline. Current LTX MLX implementations report separate maturity levels for keyframe and retake/extend paths. [S07][S08]

12.4 Duration and aspect conversion

Model constraints must not leak into timeline controls. The user requests an exact project-frame duration; the provider planner chooses legal internal dimensions/frame counts and maps the generated motion into exactly that interval.

For a two-endpoint bridge, treat the first and last generated frames as conditioning boundaries. Sample the interior at normalized positions $(j+1)/(N+1)$ for $j=0..N-1$. Store the actual generated frame sequence and the sampling map. If the native sequence length differs, only the generated interval is retimed; the original footage and speech are untouched.

Choose the model's internal duration close to the requested boundary-to-boundary interval. Do not stretch a long generated action into a tiny hold without reporting it. Legal-frame rounding, context handles, and interpolation strategy belong to the provider adapter and are covered by tests.

Preserve source framing through aspect-aware padding/cropping. For backends requiring dimensions divisible by 64, a draft such as 512×320 is legal; do not assume every nominal “512p” rectangle is legal. Process generation in an explicit SDR model color space and transform it into the project's working space afterward. Generate before editorial zoom so the same accepted artifact can support later reframing.

12.5 Motion constraints and output checks

Use a versioned short prompt template describing a locked camera, preserved composition and identity, quiet minimal motion, no speech, and no new objects. User controls expose motion amount and optional plain-language constraints, not twenty scheduler settings. Avoid invoking a large prompt-enhancement model for a fixed-purpose hold.

Do not retain generated audio. For a joint audio/video model, discarding audio output does not imply its audio computation can safely be removed; optimize that only if the implementation explicitly supports it.

Automated checks inspect endpoint discontinuity, motion magnitude, abrupt lighting changes, face/region geometry drift, and gross mouth motion where detectable. These are rejection heuristics, not guarantees of identity or silence. The user auditions every accepted generation.

Failures return a reason and retain the committed fallback. Do not replace a good freeze with a visibly broken generated face simply because inference returned a file.

12.6 Candidate lifecycle

```
Hold committed with deterministic fallback
-> request queued
-> weights verified / runtime ready
-> running
-> candidate validated
-> candidate ready for audition
-> explicit Accept
-> accepted immutable artifact becomes committed provider
```

A completed job never silently changes playback or export. Candidate acceptance is a normal undoable transaction. Reject, regenerate with a different seed, compare variants, accept, and revert to freeze are all keyboard actions.

Every job binds to Hold ID, request version, source hashes, boundary frames, and requested duration. If the hold is edited or deleted before completion, the result becomes a stale/detached candidate. It must not overwrite newer intent. Undo cancels relevance, not necessarily a running GPU kernel; workers cooperate with cancellation at safe boundaries.

12.7 Reuse and modifications

Shortening an accepted hold reuses its stored artifact and sampling map. Lengthening within valid available handles may reuse it; otherwise create a new request while keeping an explicit fallback. Changing gain, zoom, caption, or group position does not regenerate video. Changing source boundary or motion constraints does.

Preserve model/checkpoint hashes, pipeline version, prompt, seed, input frames, generation parameters, and artifact hash. A seed supports provenance and best-effort reproduction; floating-point GPU inference is not promised bit-identical across machines or runtime versions. The accepted video, not the seed, is the durable editorial truth.

13. AI implementation selection and performance qualification

13.1 Default engineering route

Implement the AI worker boundary first, then qualify two concrete candidates:

Low-cost baseline: distilled LTX-Video 2B using its supported macOS/MPS path. It is an appropriate small starting candidate for a constrained hold, not a proven fastest backend. The official project documents MPS support and image/video conditioning. [S07]

Enhanced candidate: the `dgrauet/ltx-2-mlx` implementation with a pinned compatible LTX-2.x checkpoint and keyframe/extension pipeline. Benchmark its supported precisions rather than choosing 4-bit by assumption. Its maturity document explicitly flags retake/extend concerns, so bridge quality is a qualification gate. [S08][S09]

Ship the measured winner for each supported hardware tier as the automatic default. Keep the other only when it provides a demonstrated speed/quality tradeoff worth its storage cost. Backend neutrality does not mean shipping every framework or spending the project porting models to Rust.

13.2 Alternatives considered

Candidate	Role / decision
Swift MLX LTX runtime	Benchmark if a native helper materially improves packaging or speed; do not rewrite a working Python implementation merely to remove Python.
Draw Things native inference	Valuable Metal-oriented comparison, but its GPL integration terms must be resolved before bundling. Not a hidden permissive dependency.
Wan2.2 TI2V-5B	Apache-2.0 model candidate with image-to-video support; useful license/quality alternative, not verified fast on this Mac.
Generic ComfyUI installation	Do not require it. A full external workflow server is unnecessary product and deployment machinery.
Cloud generation	Not required and not a fallback that uploads footage automatically.

The Wan model card documents 5B TI2V capability and its license. It does not establish macOS speed or endpoint-conditioned bridging. Draw Things' published licensing distinguishes a deeply integrated bundled product from an unrelated client; using a subprocess is not automatically a license exemption. [S10][S11]

13.3 Why speed cannot be guessed

One upstream Swift MLX benchmark reports a 10-second, 1024×576 image-to-video-plus-audio workload on an M3 Max taking 1,145 seconds in BF16, compared with 1,458 seconds in 8-bit and 1,294 seconds in 4-bit. This is **not** a prediction for a short hold on the user's M5 Max. It demonstrates why smaller weights must not be equated with lower latency. [S12]

Likewise, a model's advertised throughput on a datacenter GPU does not establish Mac interactivity. Measure the complete operation, including model loading, text encoding, denoising, VAE decode, validation, and artifact encoding.

13.4 Required bake-off

Use a rights-cleared fixture corpus with at least 30 short source moments: one person, two people, off-center faces, glasses, hands near faces, existing background motion, low light, compression, camera movement, and non-face details. Include 0.5, 1, 2, and 3-second holds, plus entry/exit seams.

For each hardware tier and candidate, record cold/warm latency, p50/p95 time-to-first-usable-candidate, peak unified-memory pressure, swap growth, thermal state, UI frame times during inference, output resolution, failure rate, and human acceptability. Count failed generations in performance results; timing only the successful sample is misleading.

The decision metric is **usable holds per minute under an interactive editing workload**, with identity/continuity acceptance as a constraint. Compare identical prompts, source contexts, and approximate perceptual quality, not unrelated demonstration settings.

A provisional product target is a warm, two-second draft hold within 60 seconds on the reference M5 Max. This is an unverified target, not a claimed capability. Only label a pack “Fast” after meeting its documented latency and acceptance thresholds. If no candidate meets that target, retain full AI functionality with honest measured timing and continue optimizing; do not present the freeze fallback as completed AI support.

13.5 Useful optimizations

Cache fixed text embeddings with model/tokenizer/template hashes; do not reuse image-conditioned values across different inputs. Keep a recently used model warm within a measured memory budget. Generate only the required hold and modest context, not ten seconds then throw most away. Prefer one candidate first and additional variants on demand. Use backend-supported distilled steps and fused/optimized kernels only when the quality corpus passes.

Choose precision from measured kernel performance and memory pressure. On large-memory machines, avoiding offload may matter more than minimizing weight bytes. On smaller machines, prevent swap thrashing and choose a smaller pack before trying to force a large model through aggressive streaming.

Do not repeatedly transcode full source videos for each request. Extract exact reference frames/context once and cache them. Do not run a separate general-purpose language model just to formulate “remain still.”

14. Runtime and model packaging

14.1 End-user installation contract

The standard app download contains all executable runtimes needed for supported workflows: Rust application/helpers, FFmpeg and required libraries, downloader and JavaScript support, and the chosen private model runtime. Large model weights may be installed through an in-app, resumable, verified download after the user sees the size and license.

A separate **full offline distribution** includes approved model packs as data. It requires no external installation command. “Everything bundled” does not require embedding tens of gigabytes of optional weights into every application update, but it does require that users never assemble a Python environment themselves.

If a chosen weight license requires authentication or acceptance, support the approved in-app flow and explain it. Prefer a redistributable, ungated default pack where qualification allows. Never promise anonymous downloads for gated weights.

14.2 Python isolation

Use a pinned standalone Python distribution and prebuilt, architecture-specific dependencies assembled during release builds. `python-build-standalone` is an appropriate distribution starting point. [S13]

Do not use the user's `python`, environment variables, packages, or shell initialization. No first-run `pip install`, compiler invocation, Git checkout, or dynamic import of arbitrary downloaded model code. Disable remote-code trust. Models use vetted serialization formats and safe loaders; never load untrusted pickled objects as executable code.

Run Python outside the UI process. Rust owns process lifecycle, job identity, paths, progress, cancellation, and artifact promotion. Runtime crashes cannot corrupt the project database or take down audio playback.

14.3 Model manifest

Each signed model-pack manifest includes:

```
pack_id, pack_version, model_family
checkpoint and component SHA-256 hashes
runtime_id and compatible runtime versions
license identifiers, license text, attribution, access requirements
supported operations and conditioning capabilities
legal frame-count and image-dimension constraints
supported precisions and hardware/OS requirements
estimated disk size, temporary space, memory envelope
published qualification report ID
```

The app checks free space before download/unpack, verifies hashes before activation, resumes partial downloads, and preserves a known-good pack until replacement passes a smoke test. Models are stored globally under Application Support and shared across projects. Uninstalling a pack cannot remove accepted media from a project.

14.4 Licenses are separate layers

Track runtime-code, model-weight, tokenizer/text-encoder, and converter licenses separately. LTX-Video model terms vary by checkpoint; current LTX-2.5 weights use a community license with conditions rather than a blanket Apache license. Publicly available weights are not necessarily unrestricted open-source software. [S14][S15]

The pack qualification report must identify redistribution permission, required notices, gating, and any commercial thresholds. No release may substitute a converter's MIT license for the license of the weights it converts.

15. YouTube and local import

15.1 Source import states

Import progresses through `discovered`, `downloading/copying`, `probing`, `indexed`, and `ready`, with independent analysis/proxy states. Failed imports can be retried without duplicating existing assets. Files are promoted into the asset store only after completeness and integrity checks.

Local video, audio, and still-image imports are first-class. Drag-and-drop is optional; native file dialogs and path-entry commands cover keyboard use. Use content hashes to deduplicate, not filenames.

15.2 Downloader bundle

Bundle yt-dlp plus compatible EJS support and a supported JavaScript runtime, initially Deno. Current yt-dlp documentation identifies JavaScript dependencies for full YouTube support; shipping only a yt-dlp executable and FFmpeg is not a sufficient packaging assumption. [S16]

Invoke absolute, app-controlled executable paths with argument arrays, never shell interpolation. Ignore user/global yt-dlp configuration. Disable third-party plugin discovery. Keep extractor/runtime versions visible in diagnostics. Downloader updates use app-verified signed manifests, compatibility checks, and rollback; do not let a helper overwrite itself inside the signed application bundle.

Store updated signed helpers in a controlled versioned Application Support location when necessary, validate their provenance, and keep the bundled baseline. Test this under macOS signing/notarization rules rather than treating updater execution as an ordinary file copy.

15.3 URL behavior

Accept HTTPS YouTube watch/short/share URLs, normalize the video ID, and show a title/thumbnail before expensive transfer where available. A playlist URL defaults to the indicated single video; importing a playlist requires an explicit batch action. Do not accidentally download hundreds of items.

Import the best useful original picture/audio quality, preserving source metadata; proxy decisions happen afterward. Do not transcode an original down to the current preview size. Preserve author/title/source URL, retrieval date, source identifiers, and selected stream details as private provenance metadata.

Handle deleted videos, region restrictions, rate limits, changed extractors, unavailable formats, age/auth requirements, and interrupted transfers with actionable errors. Do not promise that every YouTube URL is downloadable. Authenticated import may use an explicit user-supplied cookies file where supported; never silently read browser sessions. Treat cookies as secrets and remove temporary copies.

15.4 Security and rights

Restrict URLs and redirects to supported remote import behavior; block local-file access, private-network URL tricks, path traversal, and command-argument injection. Remote titles are untrusted display text and cannot determine filesystem paths without sanitization. Limit metadata output size and subprocess resources.

Provide a brief import notice that source-use rights remain the user's responsibility. Do not claim that remixing or parody automatically resolves copyright questions. Do not bypass DRM. No cloud service receives imported footage unless the user separately initiates a clearly specified external operation; none is needed here.

16. Media engine and native acceleration

16.1 Media abstraction

Use a narrow `MediaBackend` around demux, probing, seek/decode, frame metadata, audio decode, and encode/mux. The selected initial implementation uses pinned FFmpeg libraries with `rsmpeg` as a Rust binding candidate. Keep all unsafe and platform-specific lifetime handling inside this boundary. [S17][S18]

Maintain persistent decoder sessions, bounded decode queues, and a keyframe/PTS index. Never spawn FFmpeg for every frame step or rebuild a process-wide filter graph for every keyboard input. CLI FFmpeg remains useful for isolated background conversions and diagnostics, not the fundamental seek architecture.

16.2 VideoToolbox and Metal

Use VideoToolbox for supported hardware decode/encode and fall back to qualified software decode for other formats. Source codec, bit depth, pixel format, and device capabilities determine the actual path. Do not assume that every downloaded AV1 or unusual profile uses hardware decode. [S19]

The rendering layer consumes a `VideoSurface` abstraction with dimensions, plane layout, color metadata, PTS, and ownership. Surface representations include CPU planes, an owned native pixel buffer, and GPU textures. The baseline bounded-copy upload path must be correct before adding interop.

A native bridge may wrap `CVPixelBuffer`/`IOSurface`-backed planes in Metal textures and interoperate with `wgpu`'s Metal backend. This requires explicit synchronization, compatible texture formats/usages, retention until GPU completion, and version-specific integration. It is **not automatic zero-copy just because both sides use Metal**. Core Video provides a Metal texture-cache API, but the engine still owns these contracts. [S20]

16.3 Source formats

Support common H.264, HEVC, VP9, AV1, ProRes, AAC, PCM, MP3, Opus, PNG, and JPEG through the qualified FFmpeg build and platform codecs. Report unsupported profiles clearly. Handle VFR, interlaced sources, anamorphic pixels, rotated phone footage, mono/multichannel audio, and missing duration metadata.

Deinterlace interlaced sources before progressive presentation using a qualified deterministic path, preserving effective motion cadence when selecting the project frame rate. Avoid silently weaving combed frames. Color-range conversion distinguishes limited and full range; a missing tag triggers a documented heuristic and a correctable source interpretation, not an arbitrary global relabel.

16.4 Proxy strategy

Create proxies only when they materially improve seek or playback. Prefer a modest resolution and short GOP/intra-friendly representation appropriate to the source and disk budget. Proxies preserve frame identity/timing through a sidecar index. They are rebuildable and never replace originals in the project.

Use coarse previews immediately and refine the current frame after scrubbing settles. A stale decode completion must not overwrite the newest requested cursor frame. Proxy generation can be canceled and resumed by completed ranges; partial files are not treated as ready assets.

16.5 Realtime audio

Use CPAL or an equivalently qualified native output adapter; CPAL is the initial choice. [S21] The audio callback consumes preallocated buffers and performs no blocking I/O, allocation, locks with unbounded contention, logging, or model work. Decode and expensive DSP occur ahead of it.

The audio clock is the playback master. Display may drop stale video frames rather than drift audio. Device changes, sample-rate changes, suspend/resume, and headphone disconnects reset the transport through explicit states. Seeking flushes/reschedules audio and invalidates old video requests under a shared seek generation ID.

17. Shared render plan and evaluation

17.1 Compile structure, not a pile of clips

Compile each committed revision into an immutable `RenderPlan`. The plan provides duration lookup, active sources at a timestamp, source-time mappings, framing/effect evaluation, attached voices, and required artifacts. A balanced duration index supports seeking and ripple edits without scanning every preceding beat.

A Repeat remains a repeated subplan with occurrence parameters, not thousands of duplicated frame instructions. Plans cache structural fragments by node content hash. A small edit recompiles affected ancestors and relevant attachments, not the entire project.

17.2 One picture pipeline

```
source frame lookup / generated artifact lookup
-> source interpretation: rotation, aspect, range, transfer, primaries
-> linear working representation
-> source-local corrections / target evaluation
-> editorial crop / scale / position
-> group effects and picture attachments
-> text / simple graphics
-> output transform / tone map
-> display transform OR encoder pixel conversion
```

Use the same framing math, shaders, envelope interpolation, compositing rules, and audio graph for preview and export. They may differ in resolution or sampling quality, not in effect semantics. Do not implement every effect twice as a live wgpu shader and an unrelated FFmpeg filter expression.

Core envelopes support step, linear, and smoothstep interpolation; cubic curves are an additional explicit type with bounded control points. Default smash zoom is a step. Default creep is smoothstep. Store numeric values and curve type, not a UI animation name that might change later.

17.3 Attachment composition

Picture attachments have deterministic z-order, fit policy, and host scope. A cutaway uses full picture replacement by default; optional picture-in-picture is an ordinary rectangular overlay. A longer cutaway is trimmed to the host interval; a shorter one holds its final frame unless the user selects loop, retime, or leave-gap policy. This choice is visible in the inspector.

Text uses a small built-in set of licensed fonts and simple typography controls. A text attachment stores its actual font identity/fallback policy. Missing fonts must not silently alter line layout during export; bundled fonts guarantee the starter recipes. Subtitles are optional authored text, not always-on transcript overlays.

17.4 Stateful effects and random access

Delay, reverb, stretch, interpolation, and temporal effects require preroll or cached state. Each effect declares its latency, history length, and tail. Seeking into the middle renders enough prior state, or restores a matching state checkpoint. A one-frame preview must not sound or look different merely because playback started halfway through an effect.

Final rendering includes needed preroll but discards it outside the authored output interval. Tails extend only where the document explicitly allocates them. Warm preview quality may be reduced while scrubbing, with a visible indicator; stopped-frame inspection and export use the qualified full-quality path.

17.5 Color management

Use an explicitly defined linear-light wide-gamut working representation with named transforms; do not mix arbitrary gamma-encoded textures. Color interpretation belongs to each source. The output profile is a function of the committed document and the automatic export policy.

SDR output is Rec.709. HDR-capable paths preserve wide gamut and greater-than-reference-white values through all core effects. Tone mapping is an actual pixel transform with known metadata, never changing tags alone. Preview on an SDR display simulates/tone-maps HDR correctly and labels that viewing condition. The final automatic HDR/SDR decision is detailed in Section 22.

18. Processes, jobs, and GPU scheduling

18.1 Process responsibilities

Component	Responsibility
GUI / editor host	Command resolution, focus, document revisions, persistence, UI, job supervision.
Realtime media engine	Bounded decode, shared render plan, GPU preview, realtime audio; no Python dependency.
Native helper boundary	macOS media, texture interop, and other narrowly scoped platform functionality.
Analysis worker	ASR, VAD, shot proposals, target tracking, waveform/proxy work where isolated.
Model worker	One qualified inference backend, private runtime, validated input/output contract.
Render worker	Immutable revision rendering and encoding; can continue while the document is edited.
Download helper	Controlled remote media acquisition only.

The realtime engine may initially live in the Rust host process with bounded worker threads. Expensive or failure-prone model and final-render work is process-isolated. Do not make every tiny operation a separate microservice.

18.2 Worker protocol

Use a versioned length-framed JSON control protocol over stdio or a private Unix socket. The host sends request ID, operation, input artifact references, output workspace, constraints, revision binding, and cancellation token. Workers emit bounded progress/status messages and a final manifest. Large frames, tensors, and audio buffers are not JSON/base64 payloads.

```
{
  "protocol": 1,
  "request_id": "job-017",
  "operation": "hold.generate",
  "target": {"hold_id": "hold-4", "request_version": 3},
  "input": {"context_manifest": "inputs/context.json"},
  "output_workspace": "work/job-017",
  "constraints": {"project_frames": 45, "fps": [30, 1]},
  "provider": {"pack_id": "qualified-local-pack", "seed": 38117}
}
```

Paths in this example are scoped workspace-relative references, not arbitrary filesystem authority. The host creates the workspace and validates paths, symlinks, dimensions, media duration, and hashes before promoting outputs. Logs use stderr; stdout carries protocol messages only. Oversized or malformed messages terminate the request safely.

18.3 Job state and cancellation

Job states: queued, preflight, loading, running, validating, ready, failed, cancelling, and cancelled. A ready AI job creates a candidate; acceptance is a separate document transaction. A ready export creates a final file only after verification and atomic rename.

Persist jobs with their input hashes and target versions. On restart, abandoned workers become interrupted jobs, not successful ones. Retry reuses validated inputs and creates a new attempt ID. Cancellation first requests cooperative stop, then kills an unresponsive worker after a bounded grace period. Killing it cannot remove already accepted artifacts.

18.4 Resource priorities

Priority order: realtime audio; current-frame preview/seek; direct interactive requests; user-requested render; visible-range analysis; remaining background analysis and generation. Generation can run concurrently when it meets playback and memory budgets. Starting playback reduces inference activity at supported safe boundaries.

GPU kernels are not universally preemptible at arbitrary points. The UI must not promise instantaneous AI pause if the backend can only stop between steps. Benchmark interference and, where necessary, serialize inference with demanding preview/export work. A process-kill option remains available for a stuck worker.

A single video-generation job runs by default. Avoid loading several large models simultaneously. Budget memory from observed available memory/pressure and reserve room for the OS, preview, and other applications. A 128 GB machine is not permission to allocate 128 GB for weights and hope for the best.

18.5 Progress and power

Show actual stages, elapsed time, measured historical estimates when sufficient data exists, and an indeterminate state when progress is unavailable. Do not convert a denoising step count into total progress without accounting for loading and VAE decode.

Respect sleep, battery mode, disk pressure, thermal throttling, and application quit. A quit dialog may offer cancel jobs or keep the application running; it must not imply that work continues after every process exits. Hold generation automatically checkpoints only when its backend supports valid resumption; otherwise restarting means a new attempt.

19. Caching and incremental computation

19.1 Cache identity

Cache keys include input media hashes, stream interpretation, exact source range, normalized parameters, relevant model/checkpoint/tokenizer versions, renderer version, precision, color conversion, and output-quality tier. Keys must not be based solely on source path, modification time, or a display label.

Use BLAKE3 for internal content addressing and a conventional manifest checksum such as SHA-256 for distributed pack verification. The choice is a design decision; preserve the algorithm identifier in every hash record.

19.2 Cache categories

Category	Evictable?	Examples
Original media	No, while project references it.	Downloaded/copied source footage.
Accepted generated media	No, while any retained project revision references it.	Chosen AI hold.
Unaccepted candidates	Yes, subject to a visible retention policy.	Rejected/unused generation variants.
Derived media	Yes.	Proxies, thumbnails, waveforms, stretched-audio caches.
Analysis	Rebuildable; keep manual corrections separately.	Transcript proposals, tracking suggestions.
Final exports	User-owned files, never cache-evicted.	Rendered MP4 and report.
Model weights	App-managed installed resources.	Shared inference packs.

Project history pins all media needed for retained revisions. Cache cleanup uses reference tracking and a grace period; it cannot delete a file currently being read by a renderer or being promoted from a worker.

For durable generated masters, use a lossless FFV1/Matroska artifact with explicit pixel/color metadata, plus its generation manifest. A fast preview proxy is a separate evictable artifact. A provider may initially output another approved format, but artifact promotion performs the canonical lossless conversion once; it does not repeatedly re-encode an already accepted master. This is an internal storage choice, not another user-facing export format.

19.3 Invalidation examples

Changing a zoom invalidates relevant render fragments but not ASR or generation. Changing repeat count invalidates duration indices and affected output mappings, not the repeated source decode index. Changing a Hold's entry frame invalidates its AI request, while moving that Hold elsewhere does not. Updating a tracking correction invalidates the relevant target path, not every source in the project.

Keep caches transparent in diagnostics: key, size, origin, validity, and why a node is regenerating. Users can clear derived caches without damaging the project. Avoid hundreds of thousands of tiny per-frame files; use chunked/pyramidal containers and bounded indexes.

20. Projects, history, and recovery

20.1 Package layout

A `.deadpan` project is a macOS directory package, not a zip file edited in place.

```
Example.deadpan/
  manifest.json          # identity and lightweight discovery only
  project.sqlite         # authoritative document/history database
  Media/Originals/      # copied or APFS-cloned originals
  Media/Generated/      # accepted generation artifacts
  Media/UserAssets/     # sounds, stills, approved fonts if needed
  Analysis/Manual/      # optional portable manual sidecars
  Snapshots/            # consistent recovery checkpoints
  Reports/              # validation / provenance reports
```

Global rebuildable data lives in `~/Library/Caches/Deadpan`. Shared models and versioned helpers live in `~/Library/Application Support/Deadpan`. User keymaps/preferences live separately from project content. Export files default beside the project in an `Exports` directory, not inside the database.

`manifest.json` is not a second copy of the document. The database is authoritative. A deterministic JSON document dump is generated on demand for debugging, version review, interchange, or agent inspection.

20.2 Persistence model

Use SQLite with one writer, foreign-key checks, a tested journaling strategy, schema migrations, and atomic command transactions. Store current authored state, revision records, forward/inverse patches, asset references, manual annotations, job metadata, and named snapshots.

Commit a semantic edit durably before showing it as saved. Interactive parameter scrubbing is transient until committed and then becomes one history entry. Autosave does not mean serializing a huge file after every video frame. Batch only within a user-understandable transaction; do not silently lose a long burst of edits on crash.

Avoid two persistence models such as an event log that disagrees with a mutable JSON file. History supports undo/redo and named branches/takes, but no networked CRDT is needed. If the database uses WAL, portable snapshots/copies must include or checkpoint the WAL consistently; copying only the main file while it is open is not a valid backup procedure.

20.3 Import ownership and relinking

Default to project-managed media using an APFS clone when possible and a regular copy otherwise. Offer explicit linked-source import for large local libraries. Linked assets store path/bookmark information and content identity; moving a file invokes a relink flow.

A matching filename is insufficient for relink. Verify content hash or a deliberately defined stream fingerprint with an explicit confirmation when a source differs. Missing media appears as a placeholder with readable source identity. Do not render a final file with accidental offline placeholders unless the user explicitly authored those placeholders.

20.4 Recovery and migration

On startup, detect unclean shutdown, validate database integrity, reconcile worker attempts, and offer the latest consistent state. Take rotating consistent backups, including before a schema migration. Disk-full and permission errors stop new commits and show unsaved status; never display “Saved” after a failed transaction.

Open a newer unsupported schema read-only with an explanation rather than rewriting or discarding fields. Migrations run on a backup/copy, validate invariants, and atomically promote on success. Accepted generated media remains usable regardless of whether the current runtime can regenerate it.

20.5 Single writer and headless access

A project has one writable owner. A CLI invocation targeting an open project routes commands through the host's local authenticated socket. Otherwise it obtains a project lock and opens headlessly. Read-only render snapshots may run concurrently. Two app windows may inspect the same project but cannot independently overwrite its current state.

21. Command API, CLI, and extensibility

21.1 Pure editing core

deadpan-core knows nothing about egui, Python, GPU textures, network requests, or FFmpeg handles. It contains types, selectors, commands, validation, reduction, duration math, and serialization contracts. It emits job requests as data rather than launching processes.

Conceptual API contract:

```
fn resolve(
    document: &ProjectSnapshot,
    context: &EditContext,
    intent: &CommandIntent,
) -> Result<ResolvedCommand, EditError>;

fn apply(
    document: &ProjectSnapshot,
    command: ResolvedCommand,
) -> Result<EditTransaction, EditError>;
```

EditTransaction contains forward and inverse patches, changed IDs, anchor transformations, duration deltas, required artifacts/jobs, and a human description. Implementation details may change, but these responsibilities may not leak into UI event handlers.

21.2 JSON protocol

The external command API includes a protocol version, project ID, expected revision, command ID, semantic selector, typed parameters, and dry-run flag. Mutating requests use optimistic revision checks. A stale request returns a conflict with current revision; it does not blindly apply to whatever frame is now under the cursor.

```
{
  "protocol": 1,
  "project_id": "project-001",
  "expected_revision": "revision-031",
  "command_id": "repeat",
  "selector": {"kind": "text_object", "object": "word", "scope": "inner"},
  "parameters": {"plays": 3, "gap_frames": 0},
  "dry_run": true
}
```

A headless text-object command also supplies an explicit cursor/context or named range; do not depend on invisible GUI focus. The compact example above is valid only when executed in a session with established context.

21.3 Developer CLI

Provide a CLI shipped inside the app bundle and usable without adding it to PATH. Developer conveniences may install a symlink, but end users do not need to.

```
Deadpan --headless project validate <project>
Deadpan --headless project dump <project> --json
Deadpan --headless command <project> --json <request-file>
Deadpan --headless render <project> --output <directory>
Deadpan --headless inspect-plan <project> --frame 120
Deadpan --headless benchmark --suite <suite-manifest>
Deadpan --headless doctor
```

This is an engineering API, not a second settings-heavy editor. Headless rendering uses the same automatic output policy. Machine-readable outputs carry schema versions and structured errors. Diagnostic commands never expose cookies, tokens, or private source URLs by default.

21.4 Extension boundaries

Support declarative user recipes, named command macros, custom keymaps, and local parameter presets. These cover the primary customization need. Do not add arbitrary native plugin loading to the first complete product: it complicates signing, determinism, support, and project safety without being necessary for the stated workflow.

Backend additions implement documented media/model interfaces. New core effects require a schema addition or compatible parameter descriptor, preview/export tests, and migration rules. Existing projects remain readable even when an optional provider is unavailable.

22. Automatic YouTube-oriented rendering

22.1 User contract

One action: **Render** (Cmd-E or :render). The user chooses the destination/name if not already established, not codec, container, bit rate, GOP, pixel format, or audio settings. Show an informational summary of the automatically derived output.

Render an immutable snapshot of the **committed** document. Unaccepted AI candidates are not part of it. A pending AI Hold therefore renders its visible committed freeze/loop provider; the progress panel can say “Two holds use their current freeze versions.” No invisible regeneration, automatic candidate acceptance, or dependence on downloading a model during export.

This policy makes the export match what the editor has committed. If the user is auditioning a temporary parameter preview, Render first asks to commit or cancel that preview because it is not yet a saved revision.

22.2 Geometry and frame rate

The first primary source establishes a presentation basis: display orientation/aspect, practical native raster, and rational frame rate. Choose this automatically; inserting a sound, tiny still, or reaction later must not unexpectedly rotate or resize the whole project. The initial choice is recorded and visible.

Default raster matches the primary source's display geometry without unnecessary upscale. Normalize to square pixels and codec-legal even dimensions, keeping display aspect within a defined rounding tolerance. Letterbox/pillarbox mixed-aspect sources unless an authored framing operation fills the canvas. Do not bake player-shaped black bars around an already correct aspect ratio.

Preserve normal fractional/source frame rates. For variable-frame-rate footage, derive a stable presentation rate from analyzed timestamp cadence and keep the mapping to original PTS. For sources above 60 fps, use a supported ≤ 60 fps presentation rate chosen as an exact useful divisor where possible. Interlaced sources use the qualified deinterlace cadence. An audio-only initial project uses a 1920×1080, 30 fps black canvas. This basis is provisional only until the first time-based edit; a first primary picture can establish it beforehand. Once timed edits exist, importing picture cannot silently rebase their frame rate. Geometry can adopt the first primary picture through an explicit, previewed document transaction.

A creative `canvas` command may intentionally change framing/aspect as a document edit. It is not an export-quality dropdown. Such a change re-evaluates framing through a previewed document transaction and leaves timeline times/frame rate fixed; it never happens because a user added an unrelated source.

YouTube recommends preserving recorded frame rate and documents its upload encoding preferences. The automatic policy interprets that guidance for mixed-source edits rather than assuming every project has one unambiguous input format. [S22]

22.3 SDR output

Default SDR output:

Container:	MP4, fast-start metadata
Video:	H.264 High, progressive, 4:2:0, square pixels
Color:	Rec.709 with correct range/transfer/matrix signaling
Frame timing:	constant project frame rate, rational timestamps
Audio:	AAC-LC, 48 kHz, stereo, 384 kbit/s target
Muxing:	no edit lists; explicitly validated stream start/sync

Target two consecutive B-frames and a closed GOP around half the frame rate when the selected encoder supports those controls. These are automatic engineering settings, not user controls. Validate the actual bitstream and decoded result; do not claim the OS encoder obeys every requested property merely because a setter accepted it. YouTube documents these recommendations. [S22]

Select VideoToolbox H.264 hardware encoding first. Qualify its supported OS software fallback where available. The mandatory supported Apple Silicon release matrix must have a working encoder path; an unqualified platform must fail preflight clearly instead of silently switching to a bundled GPL encoder with unreviewed obligations.

22.4 Bitrate policy

Use these **Deadpan starting targets**, in Mbps, before a bounded source-complexity adjustment. They are deliberately simple automatic defaults, informed by rather than identical to every boundary in YouTube's published ranges. [S22]

Raster height class	≤30 fps	>30 to 60 fps
360 and below	1.5	2
480	3	4
720	5	7.5
1080	8	12
1440	16	24
2160	45	68
4320	160	240

Interpolate between classes using pixel count, accounting for non-16:9 rasters. Keep rate-control limits within the encoder's supported range. Short, high-detail or heavily transformed sections may receive extra bits within a bounded quality policy. Do not use the source bitrate directly as the quality setting; a low original bitrate is not a reason to further damage an already compressed source.

Prefer quality-constrained variable bitrate when the qualified encoder exposes reliable control; otherwise use validated target-bitrate settings. Maintain an encoded fixture suite for text edges, grain, fast cuts, zooms, and repeated compression. The goal is a high-quality upload intermediate, not the smallest possible file or a claim of one mathematically optimal universal format.

22.5 Automatic HDR branch

Produce HDR only when the picture-bearing sources are HDR, the committed effects path preserves it, and the qualified encoder supports the necessary profile and metadata. Otherwise render SDR with correct tone mapping. A mixture including SDR generated video defaults to SDR, avoiding a file labelled HDR without consistently managed HDR content. This is a deliberate automatic product policy, not a statement that mixed SDR/HDR editing is technically impossible.

For the HDR branch, use MP4 with HEVC Main10, Rec.2020 primaries/non-constant matrix, and the source-consistent PQ or HLG transfer. Preserve valid mastering metadata where appropriate and recompute content-dependent metadata when the edit changes it; do not copy stale MaxCLL/MaxFALL values blindly. YouTube documents 10/12-bit HDR, Rec.2020, PQ/HLG, and HEVC among suitable formats.

[S23]

Use a source/output-derived quality target and the qualified 10-bit encoder path. Preview uses the same branch decision as export. No source is turned into HDR merely by changing its tags, and no source is automatically upscaled to 4K to speculate about platform transcoding behavior.

22.6 Audio and mux verification

Apply authored audio and the master safety limiter, then encode AAC. Account for encoder priming, delay, B-frame timestamp reordering, and final padding. A simple `-avoid_negative_ts` flag is not a proof of correct sync. Decode the finished fixture output and check the first/last audible event against expected sample time.

Validate stream metadata, frame count, duration, monotonic presentation timestamps, orientation/aspect, color tags, channel layout, and A/V offset. The maximum allowed final A/V alignment error is one audio output sample in the internal render and a separately documented decoder/container tolerance in the encoded file; the latter must remain below one output video frame and be measured, not assumed.

For critical timing tests, include impulse audio and visible frame-number transitions at beginning, middle, and end. Verify the emitted file, not just the raw frames sent to the encoder.

22.7 Export lifecycle and provenance

Write to a uniquely named `.partial` path, complete encoding/muxing, verify, then atomically rename to the final MP4. Preserve useful resumable intermediate artifacts, but never show an incomplete file as successful. Failures include actionable diagnostic codes and retain the project unchanged.

Write a local report containing project revision, automatic policy decision, source/artifact hashes, relevant generated intervals, encoder/runtime versions, and verification results. Keep private source URLs and authentication details out of public media metadata by default.

Where realistic generated footage is present, show a nonblocking upload reminder concerning altered/synthetic-content disclosure. YouTube's disclosure rules are an upload responsibility; a local sidecar does not automatically set YouTube's disclosure flag. Do not force a visible watermark into the creative output.

[S24]

23. Repository and dependency decisions

23.1 Do not fork a whole editor by default

Decision: create Deadpan's editing core and UI in a new workspace. Reuse well-scoped libraries and selectively qualified media code. Existing timelines are not necessarily useful foundations for a structural timing editor, and an impressive feature list is not evidence that each effect works.

Repository research is a source-level fit assessment, not an execution audit or endorsement of production readiness.

Project	Relevant fit	Decision
1mrnewton/cutlass	Rust editor, native macOS media path, separately organized media/compositor components; dual MIT/Apache. Published status is early alpha, with unsettled project format and some unimplemented catalogue entries.	Best code-extraction/reference candidate. Audit decoder/encoder components; do not inherit its document model or assume all effects work.
gaussian-AI/ Gaussian_native_editor	Rust/egui/wgpu/FFmpeg architecture and project structure; additional integrations/deployment paths.	Reference patterns only unless a genuinely isolated, licensed component passes the same tests.
Niedzwiedz/ninve	Small keyboard/TUI-oriented mpv/FFmpeg trimming tool.	Too narrow as a nonlinear effects engine; do not fork.
mlm-games/Miniter	Rust backend with a different UI/platform emphasis and maintenance posture.	Not a lightweight macOS-native fit.
gyroflow/gyroflow	Relevant Rust/native GPU and media engineering; specialized stabilization product with different dependencies/license.	Study architecture where useful; do not copy code without license compatibility.

The repository descriptions and current statuses above come from the projects' own documentation. [S25][S26][S27][S28][S29]

23.2 Extraction gate

Before adopting code from Cutlass or another editor: record an exact commit and license; build it on the target Mac; run frame-accurate seeks, VFR/AAC sync, color, memory-lifetime, and export tests; inspect transitive dependencies; demonstrate that the component does not import the original application's UI/project state; and compare integration cost against the direct FFmpeg/native adapter.

Accept extraction only if it reduces complexity without requiring Deadpan to adopt foreign timeline semantics. Keep extracted code in a clearly attributed vendor module with local tests and upstream tracking. Do not create a permanent fork of a broad alpha application to avoid writing a comparatively small domain core.

23.3 Selected packages and boundaries

Area	Initial selection	Boundary / note
UI and GPU	egui, eframe, wgpu	Pin mutually compatible releases; custom rendering/accessibility tests.
Native interop	objc2 family / narrow native helper	Isolate Objective-C/Metal lifetimes; no unsafe handles in core.
Media bindings	rsmpeg and pinned FFmpeg	Wrap unstable details; verify exact build and codecs.
Audio output	cpal	Realtime callback contract and device-change tests.
Time/pitch DSP	Signalsmith Stretch through C ABI adapter	Qualify license, latency, extreme rates, and random access.

Area	Initial selection	Boundary / note
Speech analysis	whisper.cpp through C ABI or worker	Word boundaries remain correctable.
Auxiliary inference	ort where ONNX is appropriate	Provider availability is qualified, not assumed.
Persistence	rusqlite, bundled SQLite	One writer, migrations, backups, deterministic dump.
Data contracts	serde, serde_json, schemars	Version external schemas and validate inputs.
IDs / hashing	uuid or equivalent stable IDs; blake3	Persistent identity differs from content identity.
Async jobs	tokio; bounded channels	Keep async runtime and blocking work out of audio callback.
Errors / diagnostics	thiserror, tracing	Typed user-recoverable errors; redacted logs.
Paths / settings	directories, TOML configuration	No dependence on shell environment.
Testing	proptest, snapshot tests, Criterion or equivalent	Property tests plus actual media/GUI integration tests.
Downloader	yt-dlp + EJS + Deno	App-controlled, signed/pinned updater path.
Python runtime	python-build-standalone + pinned wheels	Build-time packaging, never runtime package assembly.

These are package selections, not promises about particular current version numbers. The implementation agent must lock tested versions in the initial dependency audit. The UI/media/runtime choices are documented by their respective upstream sources. [S01][S02][S03][S06][S13][S16][S17][S21][S30][S31]

24. Workspace and component contracts

24.1 Recommended layout

```

deadpan/
  crates/
    deadpan-core/      # document, time, selectors, commands, reducer
    deadpan-store/     # SQLite, history, migration, asset ownership
    deadpan-plan/      # compile document -> render plan / indices
    deadpan-media/     # FFmpeg/native media interfaces, probing, indexing
    deadpan-render/    # GPU compositor, color, visual effects
    deadpan-audio/     # clock, DSP, mixing, device adapter
    deadpan-jobs/      # supervisor, scheduling, worker protocol
    deadpan-analysis/  # analysis orchestration and annotations
    deadpan-models/    # packs, capability planning, AI requests
    deadpan-ui/        # panes, key routing, inspectors, accessibility
    deadpan-app/       # executable, lifecycle, platform integration
    deadpan-cli/       # headless command and diagnostics entry points
  native/             # narrow platform/DSP bridges only
  workers/            # private Python model worker and adapters
  recipes/            # versioned declarative starter gags
  fixtures/           # generated / rights-cleared test media
  schemas/            # command, project-dump, worker and pack schemas
  packaging/          # signed helper/model manifests and notices
  docs/               # architecture decisions, keymap, QA procedures
  xtask/              # build, bundle, verify, test orchestration

```

This is a dependency boundary map, not a requirement to create a crate for every tiny function immediately. Modules may be combined until isolation brings a clear benefit; core must remain media/UI independent. Prefer simple typed code over a generic plugin framework.

24.2 Dependency direction

core has no dependency on higher layers. store and plan depend on core. render, audio, and media consume plans/media interfaces but do not mutate documents. jobs supervises workers. models and analysis request jobs and return candidate data. ui issues commands through the application host.

The render plan should not contain a Python object, a database transaction, or a UI widget. A source asset record should not own a live decoder. A command should not perform HTTP requests while holding a project lock.

24.3 Provider traits

Define narrow interfaces for source probing/decoding, video surface upload, audio block production, inference generation, model installation, artifact storage, and encoding. Each advertises capabilities and structured failure modes. Avoid one giant Backend trait with dozens of unrelated optional methods.

Keep test doubles at these boundaries. A fake inference worker may test UI state transitions, but it does not satisfy the final AI generation requirement. A mock encoder may test job routing, but it does not satisfy playable MP4 export.

25. Performance requirements and instrumentation

These are **engineering targets**, not measured results for a program that already exists. Publish actual benchmark results with hardware, OS, build, source fixtures, power mode, and cache state.

25.1 Reference tiers

The primary optimization target is the user's M5 Max / 128 GB machine. Qualification also includes a lower-memory Apple Silicon tier, initially 16–24 GB, and an intermediate 32–64 GB tier. The smaller tier must support the complete conventional editor and a clearly reported compatible AI pack where feasible; it must not attempt to load a large pack beyond its qualified memory envelope.

Support Apple Silicon natively. Set the core application's initial deployment baseline to macOS 15, subject to dependency qualification. Model packs requiring newer macOS versions advertise that independently; do not make the whole editor require a newer OS solely because one optional backend does. Intel, Windows, Linux, and iPad ports are not required by this macOS product specification.

25.2 Targets

Measurement	Target on primary reference machine
Key event to command-state update	p95 below 8 ms while editing locally.
Cached ordinary edit to visible preview	p95 below 50 ms.
Warm seek within indexed/proxied source	p95 below 80 ms.
Cold long-GOP seek	Progressive feedback immediately; measure actual completion, target below 300 ms for the qualified common-codec suite.
Playback	Sustained 1080p60 and 4K30 for ordinary core effects at the selected preview quality.
Audio	No callback underruns in the qualified editing/inference stress suite.
Hold insertion	Committed fallback visible within 100 ms, independent of model loading.
10,000-beat project navigation	No whole-document scan per cursor movement; UI target remains unchanged.
Idle	Event-driven UI; no continuous full-frame redraw or inference loop when stopped.
AI generation	Measure the complete qualified pack path; provisional two-second draft target defined in Section 13.
Export	Measure speed by codec/effect workload; do not promise a universal real-time render rate.

When a workload cannot meet full-resolution preview, automatically reduce preview resolution or use caches while preserving exact timing. Show the quality tier. Never quietly lower final output quality merely because preview used a proxy.

25.3 Algorithmic budgets

Use indexed duration trees or a measured equivalent for seek lookup and ripple updates. Large repeat counts should have storage approximately proportional to authored structure plus sparse overrides, not the number of rendered frames. Waveforms, thumbnails, and transcript lists are virtualized.

Track decode queue depths, dropped video frames, audio underruns, render-plan compile time, GPU submission time, GPU completion latency, model memory, file I/O, and cache hit rate. Use bounded queues and backpressure everywhere. An unbounded channel is not a performance strategy.

Each performance regression should be attributable to a specific stage. Include a *doctor*/diagnostic panel that names actual active decoders, encoders, preview resolution, model pack, and fallback path without requiring the user to read logs.

26. Test strategy and fixture contracts

26.1 Core property tests

Generate valid random nested documents and commands. Verify duration conservation where required, repeat-duration equations, source-bound validity, exact frame/sample mapping, stable IDs, anchor bias, and serialization round trips. Applying a transaction and its inverse must restore an equivalent document and render plan.

Test that grouping/ungrouping and exploding a repeat preserve rendered output. Verify a moved beat keeps owned attachments, a copied beat receives new identities, and edits to one occurrence do not alter unrelated plays. Fuzz malformed schemas and command inputs without launching media workers.

26.2 Deterministic media fixtures

Generate original fixture videos with visible frame numbers, moving geometric targets, known color patches, sharp cuts, and timed audio impulses. Include integer and fractional frame rates, VFR, non-zero/negative starting PTS, B-frames, rotated video, anamorphic samples, 44.1 kHz source audio, mono/stereo/multichannel layouts, long GOPs, and interlaced material.

For AI and face tracking, use rights-cleared real-person footage with documented consent/source license. Synthetic geometric fixtures alone cannot establish identity preservation in generated human video. Keep AI quality tests separate from deterministic engine tests.

26.3 Required integration tests

Test	Required outcome
10,000 tiny fractional-rate edits	No cumulative A/V drift; expected sample/frame boundaries.
Nested repeat with gaps and one override	Exact play count, gap count, envelopes, and occurrence identity.
Mid-word silent insertion	Exactly N new frames; original speech resumes at its original source position.

Test	Required outcome
Picture-only cutaway	Host audio unchanged and attachment follows host edits.
Random seek into delay/stretch	Same state-aware output as linear playback within defined tolerances.
AI finishes after hold changes	Candidate is stale; current revision is not overwritten.
Undo during AI generation	Timeline restores; orphaned completion cannot reinsert content.
Model worker crash / OOM	Host stays responsive; project remains valid; request fails visibly.
Download interrupted / extractor changed	Retryable failure; no partial file promoted as source.
Disk fills during save or export	No false “Saved” or successful final file.
Missing / wrong relinked media	Clear placeholder/error; no silent incorrect substitution.
Proxy cleared mid-session	Correct original fallback; no source-time changes.
Export while editing	File corresponds exactly to pinned revision, not a mixture.
Accepted AI offline	Project plays/renders without its generation model installed.
SDR/HDR/rotation paths	Correct interpreted pixels and output metadata, not tags alone.
App sleep, wake, audio device switch	Recover transport without stale decode/audio state.

26.4 Visual and auditory equivalence

Compare decoded export frames against full-quality preview/reference frames after accounting for expected lossy encoding and color conversion. Define tolerances per fixture and stage. GPU implementations need not be byte-identical across devices, but geometry, timing, effect order, and content must agree.

Use lossless internal reference captures to isolate renderer bugs from codec loss. Compare audio PCM before encoding exactly where deterministic and within documented floating-point tolerances otherwise. Listen to edge cases; a numerically small error can still be an obvious click.

26.5 Keyboard and accessibility tests

Test every shipped binding, including slow prefix input, counts, Visual selections, command-line escaping, register types, macro recursion, and non-US layouts. Assert that normal-mode shortcuts never fire while editing a filename, caption, transcript, or IME composition.

Run a keyboard-only end-to-end acceptance session on a clean user account. Include model-pack acceptance/install, target selection, failed-job retry, project relink, and render destination entry. Use accessibility inspection to ensure the user can identify the selected beat and active mode without relying on color alone.

26.6 Clean-machine release test

Install the signed/notarized distribution on a supported Mac with no Homebrew, system Python setup, FFmpeg, yt-dlp, Deno, Xcode command-line tools, or preexisting model cache. Import local media, import a permitted YouTube source, download the approved model through the app, generate/accept a hold, save, restart, disconnect networking, reopen, and render.

Also test the full offline distribution from initial launch with networking disabled. Verify every nested executable/library loads under hardened runtime. A build that works only on the developer's machine does not pass.

27. Security, licensing, and distribution

27.1 Application distribution

Ship an Apple Silicon .app in a signed distribution container. Sign nested executables and libraries, enable the appropriate hardened runtime settings, notarize, and staple where applicable. Use the minimum entitlements required by the actual runtimes; do not add broad exceptions preemptively. Apple's notarization documentation is the distribution reference. [S32]

Publish checksums, third-party notices, build/toolchain provenance, and a software bill of materials. App, helper, and model updates have separate version identities and rollback rules. No surprise auto-update changes the model or renderer used by an already committed export without recording the new version.

27.2 License policy

Use a permissive license for original Deadpan code unless the owner chooses otherwise. The selected dependency build must be compatible with that decision. Keep model-weight licenses explicit and do not describe the whole distribution as permissively licensed simply because the Rust code is.

For FFmpeg, choose and record a build configuration deliberately. Optional components can change its licensing obligations. The initial plan avoids silently enabling GPL/nonfree codecs in a supposedly LGPL-oriented distribution; FFmpeg's legal page is the starting reference, not a substitute for checking the exact shipped build. [S18]

If a GPL backend such as a deeply integrated Draw Things runtime becomes the clear performance winner, make the product-license/alternative-license decision explicitly before distribution. Do not evade the issue by calling the same tightly integrated code a “helper.” [S11]

27.3 Trust boundaries

Treat imported media, project files, recipe files, remote titles, cookies, and worker outputs as untrusted. Enforce size/dimension/depth limits, bounded metadata parsing, path containment, timeouts, and safe subprocess arguments. Do not execute commands embedded in a project or recipe.

A model pack contains approved data and declared runtime compatibility, not arbitrary install scripts. Only application-signed runtime updates introduce executable code. Verify archives before extraction and reject path traversal/symlink escape. Native decoders remain a security-sensitive dependency and require timely updates.

27.4 Privacy and observability

Local-first means sources, transcripts, generated footage, and face tracks stay local. Network access is limited to user-requested imports, model/helper/app updates, and explicitly requested external links. No analytics are required; any optional diagnostics are opt-in and redact sensitive paths, source URLs, credentials, and transcript content.

The user can export a diagnostic bundle containing versions and structural error context without attaching source media. An explicit separate action may attach selected fixtures. Never include cookies or authentication headers in logs or project metadata.

28. Failure behavior and non-negotiable invariants

28.1 Invariants

1. An accepted edit has an exact authored duration and a durable revision.
2. No analysis or completed background job changes authored intent without a normal transaction.
3. Preview and export evaluate the same committed structures and effects.
4. A generated artifact's acceptance is independent of its ability to be regenerated later.
5. No user workflow depends on a manually installed runtime or a running external model server.
6. Every creative operation remains editable, undoable, serializable, and keyboard-accessible.
7. Source media and accepted generated media cannot be destroyed by cache cleanup.
8. The UI remains responsive and truthful when AI, download, decode, save, or export fails.

28.2 Structured errors

Define stable error codes with human explanations and suggested actions: `MediaUnsupported`, `MediaMissing`, `SourceRangeInvalid`, `SelectionUnavailable`, `RevisionConflict`, `ModelNotInstalled`, `ModelLicenseRequired`, `ModelUnsupportedCapability`, `InsufficientMemory`, `WorkerCrashed`, `CandidateStale`, `CandidateRejected`, `DownloadAuthenticationRequired`, `DownloadExtractorFailure`, `DiskFull`, `ProjectReadOnly`, `MigrationFailed`, and `ExportVerificationFailed`.

Errors must explain which asset/beat/job failed without exposing secrets. Preserve the user's cursor, selection, unsaved preview state where safe, and committed project. A worker's exception string is diagnostic detail, not the entire user-facing recovery experience.

28.3 Explicitly forbidden implementation shortcuts

Do not encode an entire intermediate movie after every edit. Do not store frame times as floats. Do not flatten repeats to implement their count control. Do not use one preview algorithm and a different export algorithm. Do not call a model download a successful generation. Do not swap candidates into the document automatically. Do not silently omit unsupported effects from export. Do not mark placeholder buttons, mocked workers, or ignored tests as implemented features.

29. Requirements traceability

The implementation tracker must map every requirement to code, tests, and a demonstrable acceptance result. The following IDs define the release surface; the detailed sections remain normative.

ID	Requirement	Primary evidence
DP-01	Project creation, reopen, autosave, undo/redo, migration, recovery.	Persistence/crash/migration suite.
DP-02	Exact frame/sample/source-time model including VFR.	Property tests and encoded sync fixtures.
DP-03	Structural Source/Sequence/Hold/Repeat/Retime primitives.	Golden render-plan and duration tests.
DP-04	Stable anchors, attachments, nested occurrences, single-play overrides.	Structural edit property tests.
DP-05	Complete normal/visual/operator/command/camera/trim keyboard flow.	Binding matrix and keyboard-only session.
DP-06	Registers, macros, semantic dot-repeat, configurable bindings.	Parser/transaction/replay tests.
DP-07	All time/delivery operations in Section 8.	Recipe fixture renders and editable inspector demos.
DP-08	All framing/picture operations and keyboard target selection.	Tracking/geometry/interaction tests.
DP-09	All audio operations with preserved intentional dynamics.	PCM/gain/tail/stretch fixtures.
DP-10	Local transcript, timing refinement, shot/silence proposals.	Analysis accuracy and correction tests.
DP-11	Selected target tracking with manual correction and loss handling.	Occlusion/shot-change fixtures.
DP-12	Local AI hold generation, exact seams/duration, variants, acceptance.	Actual qualified model corpus, not mocks.
DP-13	Model/runtime manager, safe downloads, offline pack installation.	Clean-machine and interrupted-install tests.
DP-14	YouTube URL import with bundled JavaScript support.	Clean-machine permitted-source import.
DP-15	Local media import, managed/linked assets, relinking.	Ownership/relink/failure tests.
DP-16	Shared realtime/offline renderer, bounded decode and proxy paths.	Preview/export comparison and stress benchmarks.
DP-17	One-action automatic SDR/HDR YouTube-oriented output.	Encoded-file metadata/pixel/sync verification.

ID	Requirement	Primary evidence
DP-18	Nonblocking worker lifecycle, cancellation, stale result handling.	Worker chaos and concurrency tests.
DP-19	Cache integrity and accepted-media portability.	Eviction/reference/offline-project tests.
DP-20	Accessible, native-behaving, simple UI.	Accessibility inspection and keyboard acceptance.
DP-21	CLI/JSON API with revision checks and dry-run.	Headless/GUI parity and conflict tests.
DP-22	Signed/notarized zero-manual-setup distribution.	Clean-machine online and offline acceptance.
DP-23	License/SBOM/privacy/security requirements.	Release audit and malicious-input tests.
DP-24	Measured performance budgets and diagnostics.	Published reproducible hardware benchmark report.

30. Implementation workstreams and delivery gates

This is an ordered full-product build plan. Work can proceed in parallel behind the stated interfaces; completing an early gate does not redefine the requested scope.

Gate A — Qualify risky dependencies

Before committing to large integration work, build and test the selected media adapter, GPU viewport path, audio output/DSP adapter, and actual model candidates on the target Mac. Audit Cutlass component extraction against the direct media adapter. Record licenses, pinned revisions, measured AI latency/quality, and packaging constraints in architecture-decision records.

Deliverables: dependency lock inventory, small isolated technical harnesses, baseline benchmark report, model-pack qualification matrix, and a source-level license inventory. No fake inference timing or assumption that macOS Python packaging can be fixed at the end.

Gate B — Establish the pure editing foundation

Implement typed time, nodes, anchors, instance paths, selectors, commands, reversible transactions, project schema, and render-plan compilation. Build the headless validator and deterministic document dump. Implement generated media fixtures and property tests before wiring UI behavior to mutable application state.

Exit criterion: representative nested edits render through a test backend with exact frame/sample selection; inverse transactions and serialization preserve meaning.

Gate C — Build the interactive media workspace

Implement actual decode/index/proxy paths, audio playback, GPU preview, basic panes, keyboard grammar, frame/word selection integration points, inspector previews, and durable history. Include native focus/IME handling and accessibility state now, not after shortcuts have become inseparable from widgets.

Exit criterion: edit and audition real footage through keyboard commands with measurable latency and no drift. This is an integration checkpoint, not the finished product.

Gate D — Complete the creative operation surface

Implement every operation and starter recipe in Section 8, including per-play overrides, tails, pitch/stretch, cutaways, target framing, saved gags, registers, and semantic macros. Validate each in both preview and final rendering. Populate the command/help schema from the same registry used by the parser.

Exit criterion: the operation matrix has no no-op placeholders, and recipe expansions remain editable and portable.

Gate E — Add analysis and real AI holds

Integrate local transcription/VAD/shot proposals, confidence and manual corrections, keyboard target selection/tracking, runtime/model manager, generation planning, validation, candidate audition/acceptance, stale-job handling, and caching. Run the full real-video generation corpus on the supported hardware tiers.

Exit criterion: actual local generations meet documented duration/seam contracts and produce qualified usable results; accepted projects render offline without the model. Publish measured latency rather than substituting a generic “AI enabled” badge.

Gate F — Complete import, export, and distribution

Integrate yt-dlp/EJS/Deno, source provenance, safe updates, automatic render policy, HDR/SDR color tests, codec/mux verification, licensing notices, signed runtime bundles, notarization, and clean-machine installation. Finish migration/recovery and disk-full/permission flows.

Exit criterion: the full keyboard-only source-URL-to-final-MP4 workflow runs from the distributed application with no external setup.

Gate G — Release qualification

Run the requirements matrix, crash/chaos tests, malicious-input suite, long-project stress tests, preview/export comparisons, online/offline installers, and accessibility checks. Measure all performance targets and document any remaining deviations with severity and user-visible behavior.

The project may not be declared complete while required operations are stubs, AI is simulated, export ignores effects, or end users must install dependencies. A release candidate includes the app, approved packs, documentation, fixture reports, benchmark reports, SBOM/notices, and migration policy.

31. End-to-end worked edit

This example uses a generic interview clip. It specifies expected semantics, not a particular person's behavior.

1. Import the clip, insert a source range into the sequence, and let transcription analyze it locally.
2. Search for an ordinary word such as “absolutely.” Navigate to its occurrence and type `3riw`. The selected word's enclosing frame range becomes a Repeat with three total plays and no trailing gap.

3. With the Repeat selected, enter `:repeat 3 gap=120ms gain-step=3dB zoom-step=0.08`. The existing repeat's parameters are updated, not nested into another three-by-three repeat. An explicit `wrap-repeat` command is used for intentional nesting of an already selected Repeat.
4. Navigate to the end of the sentence and insert `3,h`. A 1.5-second freeze with silence is added at the cursor boundary. The next sentence shifts later but retains its source timing.
5. Select the Hold, enter Camera mode, choose a numbered face/region, and set a slow creep from 1.0× to 1.35×. The Hold remains 1.5 seconds long.
6. Convert its visual request to AI using `:hold-provider ai`. This requests a candidate for the existing Hold without inserting another hold. Audition it and explicitly accept. The creep is applied after the generated source picture.
7. Yank a reaction into register `r` from the source browser. Select a portion of the hold and use `:cutaway register=r audio=keep`. That picture replaces the chosen interval while the selected silence/tail policy remains unchanged.
8. Group the result as “the long answer,” save it as a local gag recipe, and adjust the overall hold duration by frames. No effect is baked into a new monolithic source clip.
9. Render. The output uses the current committed version, automatic source-derived geometry/frame rate, and the appropriate SDR/HDR branch. The report identifies the accepted generated interval.

31.1 Command targeting clarification

When the selected node already has the same operation type, the command-line setter edits that node's parameters. `:repeat` updates a selected Repeat; `:hold-duration` updates a selected Hold. In contrast, an operator on a selected range wraps that range, and `:hold` always inserts new time. The palette descriptions must distinguish **Insert hold**, **Change hold duration**, **Change hold provider**, **Wrap repeat**, and **Set repeat parameters**.

This avoids a common usability failure: a user intending to change three plays to four accidentally creating a nested repeat. The command registry contains separate internal command IDs even when friendly syntax shares a word.

31.2 Source-context behavior

Source browsing is non-destructive. Navigation, marking, searching, and yanking work directly against the source. An editing operator such as `r` or `d` in Source context does not alter the original or silently create a new timeline. It offers the already-defined action **Insert selected range into sequence**; the user can invoke that with `:insert` or its visible palette binding. Once inserted, editing takes place in Sequence context.

The source browser can preview trims, create named ranges, and set register contents without committing them to the sequence. Make the current context prominent so the same letter cannot appear to delete an original file.

32. Resolved choices and remaining empirical questions

The product concept, editing primitives, keyboard language, project ownership model, runtime boundary, packaging requirement, and automatic rendering policy are decided in this document. They should not be repeatedly reopened by implementation agents without evidence of a real conflict.

Three areas require measurement rather than invented certainty: which local model/backend gives the fastest acceptable holds on the reference Mac; whether selected upstream media components are cheaper to qualify than direct adapters; and which native encoder/texture interop configurations pass the exact deployed OS matrix. These are concrete Gate A tasks with acceptance criteria, not unspecified product features.

The main engineering priority is to preserve the user's timing decisions. A one-frame error, an automatic gain normalizer, an invisible generated replacement, or a flattened repeat can damage the intended result even when the application appears technically sophisticated.

The completed product should make “hold that face for another eleven frames” a precise, immediate edit—and make everything surrounding that decision stay out of the way.

33. Primary sources and research notes

All sources below were consulted for the 20 September 2026 design. Project/model status can change; pin and qualify exact revisions during implementation. Source references support external facts, while architecture, defaults, thresholds, and UX rules are Deadpan design decisions. No third-party benchmark is a measurement of Deadpan.

- **[S01] egui / eframe.** Native integration, wgpu backend, accessibility integration. <https://github.com/emilk/egui>
- **[S02] Signalsmith Stretch.** Author's time/pitch-stretch library documentation. <https://signalsmith-audio.co.uk/code/stretch/>
- **[S03] whisper.cpp.** Runtime capabilities and experimental word-level timestamps. <https://github.com/ggml-org/whisper.cpp>
- **[S04] Silero VAD.** Local speech-activity detection implementation. <https://github.com/snakers4/silero-vad>
- **[S05] Apple Vision face tracking.** Native tracking reference. <https://developer.apple.com/documentation/vision/tracking-the-user-s-face-in-real-time>
- **[S06] ort.** Rust ONNX Runtime adapter. <https://github.com/pykeio/ort>
- **[S07] Lightricks LTX-Video.** Distilled model choices, macOS/MPS support, conditioning/extension workflows. <https://github.com/Lightricks/LTX-Video>
- **[S08] LTX MLX pipeline maturity.** Upstream maturity classifications and extension/retake limitations. https://github.com/dgrauet/ltx-2-mlx/blob/main/docs/PIPELINE_MATURITY.md
- **[S09] dgrauet LTX-2 MLX.** Native MLX implementation and supported model/pipeline options. <https://github.com/dgrauet/ltx-2-mlx>
- **[S10] Wan2.2 Ti2V-5B model card.** Image/video capability, size, and model license. <https://huggingface.co/Wan-AI/Wan2.2-Ti2V-5B>
- **[S11] Draw Things community inference.** Native implementation, CLI, and integration licensing. <https://github.com/drawthingsai/draw-things-community>
- **[S12] Swift MLX LTX implementation.** Published M3 Max benchmark and precision comparison. <https://github.com/VincentGourbin/ltx-video-swift-mlx>
- **[S13] python-build-standalone.** Standalone Python distribution tooling. <https://github.com/astral-sh/python-build-standalone>
- **[S14] LTX-Video model card.** Checkpoint-specific weight terms and generation constraints. <https://huggingface.co/Lightricks/LTX-Video>
- **[S15] LTX-2.5 model card.** Current weight access and community-license conditions. <https://huggingface.co/Lightricks/LTX-2.5>
- **[S16] yt-dlp.** Distribution, JavaScript/EJS requirements, runtime support, and downloader options. <https://github.com/yt-dlp/yt-dlp>
- **[S17] rsmpeg.** Rust FFmpeg binding project. <https://github.com/larksuite/rsmpeg>
- **[S18] FFmpeg legal information.** License configuration and distribution obligations. <https://ffmpeg.org/legal.html>
- **[S19] Apple VideoToolbox.** Native video compression/decompression framework. <https://developer.apple.com/documentation/videotoolbox>

- **[S20] Apple CVMetalTextureCache.** Core Video / Metal texture integration primitive. <https://developer.apple.com/documentation/corevideo/cvmetaltexturecache>
- **[S21] CPAL.** Native cross-platform audio output/input library. <https://github.com/RustAudio/cpal>
- **[S22] YouTube recommended upload encoding settings.** Upload container, codec, frame rate, audio and bitrate guidance. <https://support.google.com/youtube/answer/1722171?hl=en>
- **[S23] YouTube HDR uploads.** HDR transfer, primaries, metadata and encoding requirements. <https://support.google.com/youtube/answer/7126552?hl=en>
- **[S24] YouTube altered/synthetic-content disclosure.** Upload disclosure requirements and examples. <https://support.google.com/youtube/answer/14328491?hl=en>
- **[S25] Cutlass.** Current product scope, alpha status, platform media backend, and license. <https://github.com/1mrnewton/cutlass>
- **[S26] Gaussian native editor.** Rust editor architecture and integrations. https://github.com/gaussian-AI/Gaussian_native_editor
- **[S27] Ninve.** Keyboard/TUI trimming scope and dependencies. <https://github.com/Niedzwiedz/ninve>
- **[S28] Miniter.** UI/backend/platform scope and maintenance information. <https://github.com/mlm-games/Miniter>
- **[S29] Gyroflow.** Rust/native media architecture and license. <https://github.com/gyroflow/gyroflow>
- **[S30] objc2.** Rust Objective-C interoperability documentation. <https://docs.rs/objc2/latest/objc2/>
- **[S31] wgpu.** Graphics API and backend documentation. <https://docs.rs/wgpu/latest/wgpu/>
- **[S32] Apple notarization.** Distribution and notarization workflow. <https://developer.apple.com/documentation/security/notarizing-macos-software-before-distribution>

Additional implementation references

FFmpeg's filter and encoder documentation is useful for diagnostic/conversion helpers and capability checks, but it does not replace the shared Deadpan renderer contract: <https://ffmpeg.org/ffmpeg-filters.html> and <https://github.com/FFmpeg/FFmpeg/blob/master/libavcodec/videotoolboxenc.c>.

The current official LTX-2 implementation provides pipeline/model context when qualifying a modern backend: <https://github.com/Lightricks/LTX-2>. Follow the selected checkpoint's actual license and pipeline support rather than inferring compatibility from a shared family name.